

EC2-instance-type-pricing

Amazon Elastic Compute Cloud (Amazon EC2)

EC2 is basically a virtual machine in AWS.

Instead of buying and maintaining physical servers, AWS provides you with virtual servers that you can create, scale, and terminate on demand

You don't manage hardware

AWS handles:

Physical servers

Data centers

Networking infrastructure

EC2 is part of IaaS (Infrastructure as a Service), where AWS provides compute infrastructure, and the user controls the operating system and software stack.

A traditional virtual machine runs on on-premise infrastructure where

the organization is responsible for managing the physical hardware, virtualization layer, and maintenance.

Traditional VM (onpremise)

You manage:

Physical servers

Hypervisor (VMware, etc.)

Networking

Power, cooling

EC2

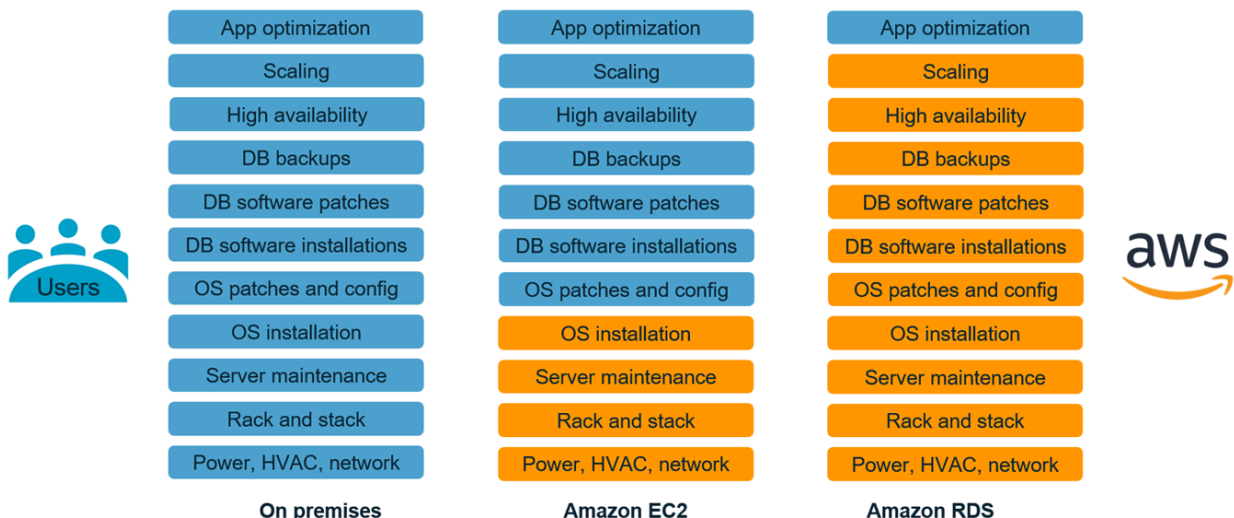
You only manage:

Os

Application

configuration

AWS manages: Hardware Data centers Networking infrastructure



In contrast, EC2 is a cloud-based virtual machine where AWS manages the underlying hardware, networking, and data center operations, and the user only manages the operating system and applications.

The main difference is that EC2 abstracts away infrastructure management, while traditional VMs require you to manage the entire underlying hardware stack.

EC2 User Data: (Bash Script)

EC2 User Data is a script that runs automatically when an EC2 instance is launched, used to bootstrap and configure the instance.

EC2 User Data allows you to run a script at the time of instance startup to automate tasks like installing software, updating packages, or configuring applications.

Key Behavior (Important Detail)

Runs only once at the first launch (by default)

Executes with root privileges

Used for initial setup automation

For example:

I can use User Data to install a web server like Nginx and deploy an application automatically when the instance starts.

Why It Matters

Eliminates manual setup

Enables automation

Useful in:

Auto Scaling

Infrastructure as Code

CI/CD pipelines

How Naming Works (Important)

EC2 instance types define the compute capacity of a virtual machine, including CPU, memory, storage, and networking, optimized for different workloads.

Example1:

t2.micro
t → instance family
2 → generation
micro → size

Example2 :

g4dn.xlarge
g → family
4 → generation
xlarge → size

Example3:

t3.medium
t → instance family
3 → generation
medium → size

EC2 Instance Types:

Instance Families

1. General Purpose (Balanced):

General purpose instances provide a balance of compute, memory, and networking, suitable for most applications.

Families:

T family (t2, t3, t3a)

M family (m5, m6)

T Family (Burstable Instances)

Key Concept:

Baseline performance + burst capability using CPU credits

Use Cases:

Small applications

Dev/Test environments

Low traffic web servers

Scenario 1: Startup Web Application (Low Traffic + Spikes)

Situation

- Small startup
 - Web app with unpredictable traffic
 - Budget is tight
-

Solution

- **Instance Type:** T family (e.g., t3.micro)
 - **Pricing:** On-Demand
-

Why

- T family → burstable CPU (handles spikes)
 - On-Demand → no commitment (traffic unpredictable)
-

Interview Answer

For a low-traffic application with occasional spikes, I would choose a T-family instance with On-Demand pricing because it provides cost efficiency with burst capability and flexibility.

2. Compute Optimized:

Compute optimized instances are designed for CPU-intensive workloads requiring high processing power.

Families:

C family (c5, c6)

Use Cases:

CPU-intensive workloads:

Batch processing

Media encoding

High-performance web servers

Machine learning inference

Interview Line:

Scenario 2: Production Backend API (Stable Traffic)

Situation

- API running 24/7
 - Predictable load
 - Business-critical
-

Solution

- **Instance Type:** C family (C5.large)
 - **Pricing:** Reserved Instances / Savings Plan
-

Why

- C family → balanced performance
 - Reserved → cost savings for steady usage
-

Interview Answer

For a stable production workload, I would use C-family instances with Reserved Instances or Savings Plans to reduce cost while maintaining consistent performance.

3. Memory Optimized

Memory optimized instances are used for applications that require high memory performance, such as databases and caching systems.

Families:

R family (r5, r6)

X family

Use Cases:

High memory workloads:

Databases (SQL/NoSQL)

In-memory caching (Redis)

Real-time analytics

Interview Line

Scenario 4: High-Performance Database

Situation

- Large database (production)
 - Needs high memory
 - Must be reliable
-

Solution

- **Instance Type:** R family (r5 / r6)
 - **Pricing:** Reserved Instances
-

Why

- Memory optimized → database performance
 - Reserved → long-term cost saving
-

Interview Answer

For a production database, I would choose memory-optimized instances with Reserved pricing to ensure performance and cost efficiency

4. Storage Optimized

Storage optimized instances are designed for workloads requiring high read/write performance on local storage.

Families:

I family

D family

Use Cases

High disk throughput:

Data warehousing

Big data processing

OLTP systems

Interview Line

5. Accelerated Computing:

Accelerated computing instances use specialized hardware like GPUs and FPGAs to speed up workloads such as machine learning, graphics rendering, and scientific computing.

Families:

P → GPU (machine learning training)

G → GPU (graphics + ML inference)

F → FPGA (custom hardware acceleration)

Inf → AWS Inferentia (ML inference optimized)

What is Accelerated Computing?

These instances use hardware accelerators like GPUs or FPGAs to perform specialized tasks much faster than CPUs.

Use Cases

Machine Learning:

Training models → P family

Inference → G / Inf

Graphics:

Video rendering

Game streaming

High-performance computing:

Scientific simulations

Interview Line

Scenario 5: Machine Learning Training

Situation

- Training ML models
 - Heavy GPU usage
 - Runs for limited time
-

Solution

- **Instance Type:** P family (GPU)
 - **Pricing:** Spot Instances
-

Why

- GPU → faster training
 - Spot → reduce high cost
-

Interview Answer

For ML training, I would use GPU-based instances with Spot pricing to optimize both performance and cost.

Scenario 6: Video Streaming / Rendering

Situation

- Media processing
 - GPU-intensive
-

Solution

- **Instance Type:** G family
 - **Pricing:** On-Demand / Spot
-

Why

- GPU required
 - Choose pricing based on reliability needs
-

Scenario 7: Big Data / Hadoop Cluster

Situation

- Distributed system
 - Large storage + throughput
-

Solution

- **Instance Type:** I or D family
 - **Pricing:** Spot + Reserved mix
-

Why

- Storage optimized
 - Hybrid pricing reduces cost
-

Scenario 8: Dev/Test Environment

Situation

- Used only during working hours
 - Not critical
-

Solution

- **Instance Type:** T family
 - **Pricing:** On-Demand + Stop/Start strategy
-

Why

- Cheap
 - Can shut down when not needed
-

AWS EC2 Pricing Options – Interview Ready Notes

Definition:

EC2 pricing options define different ways to pay for compute capacity based on workload requirements, cost optimization, and commitment level.

AWS provides multiple pricing models for EC2 so that you can choose between flexibility, cost savings, or reliability depending on your workload.

3. Core Pricing Models

1. On-Demand Instances:

On-Demand instances are used when workloads are unpredictable and require maximum flexibility without commitment.

Definition:

Pay for compute by usage (per second/hour)

No long-term commitment

Key Features:

Highest cost

No upfront payment

Fully flexible

2. Reserved Instances (RI)

Reserved Instances are ideal for predictable workloads where long-term commitment reduces cost significantly.

Definition:

Commit for 1 or 3 years to get significant discount

Key Features:

Up to ~72% cheaper than On-Demand

Fixed configuration

Payment options:

No upfront

Partial upfront

All upfront

Types:

Standard RI

Convertible RI (flexible instance type)

3. Savings Plans:

Savings Plans provide cost savings similar to Reserved Instances but with more flexibility across instance types.

Definition: Commit to a fixed usage (\$/hour) for 1 or 3 years

Key Features:

Flexible across:

Instance size

OS

Similar savings as Reserved Instances

4. Spot Instances

Definition:

- Use unused AWS capacity at very low cost
-

Key Features:

- Up to ~90% cheaper
 - Can be terminated anytime
-

Use Cases:

- Batch processing
 - Data analysis
 - Fault-tolerant workloads
-

Not Suitable For:

- Critical applications
 - Databases
-

Interview Line:

Spot Instances are cost-effective but unreliable, suitable only for fault-tolerant workloads.

4. Advanced Pricing Options

5. Dedicated Hosts

Definition:

- Physical server fully dedicated to you
-

Key Features:

- Full hardware control
 - Most expensive
-

Use Cases:

- Compliance requirements
 - Bring Your Own License (BYOL)
-

6. Dedicated Instances

Definition:

- Instances run on dedicated hardware (not shared with other customers)
-

Key Features:

- Less control than Dedicated Hosts
 - Hardware isolation
-

Use Cases:

- Security-sensitive workloads

7. Capacity Reservations

Definition:

- Reserve EC2 capacity in a specific Availability Zone
-

Key Features:

- No discount
 - Guaranteed capacity
-

Use Cases:

- Critical workloads needing guaranteed availability

5. Quick Comparison (Important for Interviews)

Pricing Model	Cost	Flexibility	Reliability
On-Demand	High	High	High
Reserved Instances	Low	Low	High
Savings Plans	Low	Medium	High
Spot Instances	Very Low	Low	Low
Dedicated Hosts	Very High	Low	Very High

6. Real Interview Insight (Most Important)

Choosing pricing is about:

Cost vs Reliability

Predictability of workload

Risk tolerance

7. One-Line Strong Answer

EC2 pricing options like On-Demand, Reserved Instances, Savings Plans, and Spot Instances allow users to balance cost, flexibility, and reliability based on workload requirements.

Key-Pair

What is a Key Pair in AWS EC2?

Definition:

A key pair in AWS consists of a public key and a private key used for secure authentication to EC2 instances instead of using passwords.

Simple Explanation

- Public key → stored in EC2
- Private key → downloaded by you (just like password)

When you connect (SSH), AWS verifies your private key against the public key.

A key pair is used to securely log into EC2 instances using cryptographic authentication instead of passwords.

Why Key Pair is Important

- No password login → more secure
- Prevents brute force attacks
- Standard practice in DevOps

.pem vs .ppk (VERY IMPORTANT INTERVIEW QUESTION)

What is .pem?

Definition

- Privacy Enhanced Mail format
 - Standard private key format used in Linux/Mac
-

Used With

- SSH (Linux/Mac terminal)
-

Example

`ssh -i mykey.pem ec2-user@<ip>`

What is .ppk?

Definition

- PuTTY Private Key (ppk) format
-

Used With

- PuTTY (Windows SSH client)

Key Difference

Feature	.pem	.ppk
Format	Standard private key	PuTTY-specific format
Used In	Linux / Mac (SSH)	Windows (PuTTY)
Default from AWS	Yes	No
Conversion Needed	No	Yes (from .pem)

3. Conversion (Real Practical Point)

If using Windows:

- AWS gives `.pem`
- You convert to `.ppk` using:
 - PuTTYgen

Interview Answer

The `.pem` file is the default private key format provided by AWS and is used with SSH on Linux or Mac systems, while `.ppk` is a PuTTY-specific format used on Windows. If needed, `.pem` can be converted to `.ppk` using PuTTYgen.

5. Advanced Insight (This is where you stand out)

- Both `.pem` and `.ppk` are just **different formats of the same private key**
- Security depends on:
 - Keeping the private key safe
- If lost → you cannot SSH into instance

Ec2-lifecycle

EC2 Instance Lifecycle (Interview Ready + Real-World Use Cases)

1. Overview

EC2 instances go through different states during their lifecycle, and each state is used based on operational or cost requirements.

2. Instance States & Actions

1. Start

What is it?

- Moves an instance from **stopped** → **running**
-

Key Points

- New public IP (if not using Elastic IP)
 - EBS volumes remain attached
 - OS boots normally
-

Real-World Use Case

- Dev/Test environments:
 - Start instances only during working hours
 - Cost saving strategy:
 - Stop at night, start in morning
-

Interview Line

Start is used to bring a stopped instance back to running state without losing configuration.

2. Stop

What is it?

- Shuts down the instance (graceful shutdown)
-

Key Points

- Instance is not running
 - No compute charges
 - EBS data persists
 - Instance store data is lost
-

Real-World Use Case

- Non-production workloads:
 - QA environments
 - Training labs
 - Cost optimization
-

Interview Line

Stop is used to temporarily halt an instance while preserving data and configuration.

3. Reboot

What is it?

- Restart the instance (like OS reboot)
-

Key Points

- Instance stays on same host
 - No data loss
 - No billing interruption
-

Real-World Use Case

- Apply OS updates
 - Fix minor issues
 - Restart services
-

Interview Line

Reboot is used to restart the operating system without affecting the instance configuration.

4. Terminate

What is it?

- Permanently deletes the instance
-

Key Points

- Instance is destroyed
 - Cannot be recovered
 - EBS may be deleted (based on settings)
-

Real-World Use Case

- Auto Scaling:
 - Replace unhealthy instances
 - Cleanup:
 - Remove unused resources
-

Critical Insight

- This is **destructive and irreversible**
-

Interview Line

Terminate is used to permanently delete an instance when it is no longer needed.

5. Hibernate

What is it?

- Saves RAM state to disk and stops the instance
-

Key Points

- Resume from same state
 - Faster than start
 - Requires:
 - EBS-backed instance
 - Supported instance type
-

Real-World Use Case

- Long-running processes
 - Applications with in-memory state
 - Development environments
-

Interview Line

Hibernate preserves the in-memory state of an instance, allowing it to resume exactly where it left off.

3. Quick Comparison (Interview Shortcut)

Action	Data (EBS)	RAM State	Billing	Use Case
Start	Preserved	Lost	Charged	Resume work
Stop	Preserved	Lost	Not charged	Cost saving
Reboot	Preserved	Lost	Charged	Restart OS
Terminate	Deleted	Lost	Stopped	Cleanup / scaling
Hibernate	Preserved	Saved	Minimal	Fast resume

4. Real DevOps Thinking (This is what matters)

- Use **Stop** → save cost
 - Use **Terminate** → stateless design
 - Use **Hibernate** → stateful workloads
 - Use **Reboot** → troubleshooting
-

5. Common Mistakes (You must avoid)

- Thinking stop = terminate
- Not understanding data loss scenarios
- Ignoring instance store vs EBS

6. Strong One-Line Answer

EC2 lifecycle actions like start, stop, reboot, terminate, and hibernate allow flexible management of compute resources based on cost, performance, and application requirements.

ENI

EC2 Network Interfaces (ENI) – Interview Ready

1. What is a Network Interface (ENI)?

Definition (Interview Ready)

An Elastic Network Interface (ENI) is a virtual network card attached to an EC2 instance that enables network connectivity within a VPC.

2. Simple Explanation

Think of ENI as:

The **network adapter (NIC)** of your EC2 instance

Just like a physical server has a network card, EC2 has ENIs.

3. What Does an ENI Contain?

An ENI includes:

- Private IP address
 - Public IP (optional)
 - Elastic IP (optional)
 - MAC address
 - Security groups
 - Network interface ID
-

4. Why is ENI Important?

Core Reasons

- Enables communication inside VPC
 - Controls network access via security groups
 - Allows multiple IPs per instance
 - Can be attached/detached dynamically
-

Real DevOps Use Case

- Move ENI from one instance to another
→ Failover setup

Example:

- Instance A fails
 - Attach ENI to Instance B
 - Traffic continues
-

Interview Line

ENI provides flexible networking by allowing dynamic attachment, IP management, and security configuration for EC2 instances.

5. Types of ENI

- Primary ENI (eth0)
 - Created automatically
 - Cannot be detached
 - Secondary ENI
 - Can be attached/detached
 - Used for advanced networking
-

6. What Happens When You Delete an ENI?

Case 1: Primary ENI

- Cannot be deleted directly
 - Deleted automatically when instance is terminated
-

Case 2: Secondary ENI

If you delete it:

- Instance loses:
 - Associated IP addresses
 - Network connectivity via that interface
 - Any services using that ENI will break
-

Real Impact

- App downtime
 - Lost traffic routes
 - Broken connections
-

7. Critical Insight (Interview Level)

- ENI = identity in network
 - Deleting ENI = removing network identity
-

8. Common Mistake

People think:

“ENI is just IP”

Wrong.

It's:

- IP + security + identity + routing context
-

9. One-Line Strong Answer

An ENI is a virtual network interface that provides IP addressing, security groups, and connectivity for an EC2 instance, and deleting it removes the network access associated with that interface.

Security Groups

AWS Security Groups – Interview Ready

1. What is a Security Group?

Definition (Interview Ready)

A Security Group is a virtual firewall attached to an EC2 instance that controls inbound and outbound traffic.

2. Simple Explanation

It acts like a **firewall at the instance level**, deciding:

- Who can access your instance
 - What traffic is allowed
-

3. Default Behavior (VERY IMPORTANT)

Inbound Traffic

-  Denied by default
-

Outbound Traffic

-  Allowed by default
-

Interview Line

By default, Security Groups deny all incoming traffic and allow all outgoing traffic.

4. Key Characteristics

- Attached to **EC2 instances**
 - Works at **instance level**
 - Supports:
 - IP-based rules
 - Port-based rules
 - Can attach multiple security groups to one instance
-

5. What Does Stateful Mean? (CRITICAL)

Definition

A stateful firewall automatically allows return traffic for an allowed request, regardless of outbound or inbound rules.

Simple Explanation

If you allow inbound traffic:

- The response is automatically allowed
 - 👉 You don't need to create a separate outbound rule
-

Example

You allow:

- Inbound: Port 22 (SSH)

Then:

- Outbound response is automatically allowed
-

Interview Line

Security Groups are stateful, meaning if a request is allowed in one direction, the response traffic is automatically allowed.

6. Why Stateful Matters

Without stateful behavior:

- You would need:
 - Separate inbound rule
 - Separate outbound rule
-

Real Impact

- Simplifies configuration
 - Reduces errors
 - Improves usability
-

7. Real-World Use Cases

Web Server

- Allow:
 - Port 80 (HTTP)

- Port 443 (HTTPS)
-

SSH Access

- Allow:
 - Port 22
 - Only from your IP
-

Database Security

- Allow:
 - DB port (e.g., 3306)
 - Only from application server SG
-
-

8. Advanced Concepts

1. Referencing Security Groups

Instead of IP:

- Allow traffic from another Security Group

👉 More secure + dynamic

Use Case

- App server → DB server
-
-

2. Security Groups vs NACL

Feature	Security Group	NACL
Level	Instance	Subnet
Type	Stateful	Stateless
Rules	Allow only	Allow + Deny

3. No Explicit Deny

Security Groups:

- Only allow rules
 - Everything else → denied by default
-
-

4. Multiple Security Groups

- Rules are combined (union)
 - No conflict — only allowed traffic passes
-
-

9. Common Mistakes (Interview Killers)

- Thinking SG is stateless ✗
 - Forgetting default outbound allow ✗
 - Using 0.0.0.0/0 everywhere ✗
-

10. One-Line Strong Answer

Security Groups are stateful instance-level firewalls that control inbound and outbound traffic, with default inbound deny and outbound allow behavior.

EC2-storage

1. What is a File System?

Definition (Interview Ready)

A file system is a method used by an operating system to organize, store, and manage data on a storage device like a disk or EBS volume.

Simple Explanation

Think of it like:

👉 **How data is structured on disk**

Without a file system:

- Disk = raw blocks
 - OS cannot understand data
-

Real Analogy

- EBS Volume = empty hard disk
 - File system = way to organize files (like folders, indexes)
-

Key Point

👉 When you attach an EBS volume, you must:

1. Format it (create file system)
 2. Mount it
-

2. Why File System is Needed in EBS

From your notes :

- EBS is a **block storage (raw disk)**
- It doesn't know:
 - files
 - folders

👉 OS needs file system to use it

3. Common File Systems (Interview Focus)

1. FAT (File Allocation Table)

What is it?

- Old, simple file system
-

Types

- FAT16
 - FAT32
-

Characteristics

- Simple
 - Compatible with almost all OS
-

Limitations

- File size limit (~4GB in FAT32)
 - Not efficient for large systems
-

Use Cases

- USB drives
 - Small storage devices
-

Interview Line

FAT is a simple and widely compatible file system, mainly used for small storage devices.

2. NTFS (Windows File System)

What is it?

- Default file system for Windows
-

Features

- Large file support
 - Security (permissions)
 - Journaling (data recovery)
-

Use Cases

- Windows servers
 - Enterprise systems
-

Interview Line

NTFS is a robust file system used in Windows that supports large files, security permissions, and reliability features.

3. EXT (Linux File System)

Types

- ext3
 - ext4 (most common)
-

Features

- Journaling
 - High performance
 - Stable
-

Use Cases

- Linux EC2 instances
 - Most AWS workloads
-

Interview Line

ext4 is the most commonly used file system in Linux, providing stability, performance, and journaling capabilities.

4. XFS

What is it?

- High-performance file system
-

Features

- Handles large data
- High throughput

Use Cases

- Big data
 - High I/O workloads
-

Interview Line

XFS is used for high-performance workloads requiring fast read/write operations on large datasets.

4. Practical Flow (VERY IMPORTANT)

When you attach EBS:

Step 1: Check disk

lsblk

Step 2: Create file system

mkfs -t ext4 /dev/xvdf

Step 3: Mount

mount /dev/xvdf /data

Step 4: Use it

Now OS understands:

- files
- directories

5. Real DevOps Insight

- 👉 EBS = raw storage
- 👉 File system = usable storage

Without file system:

- You cannot store files properly

6. Common Mistakes

- Thinking EBS already has file system ❌
- Forgetting to format disk ❌
- Using wrong file system for workload ❌

7. One-Line Strong Answer

A file system defines how data is stored and organized on a disk, and in AWS, it is required to make EBS volumes usable by the operating system.

Storing MySQL Logs on a Separate EBS Volume

Production Scenario: Storing MySQL Logs on a Separate EBS Volume

1. Scenario Overview

In a production environment, MySQL logs continuously grow as the database handles queries (read/write/update/delete).

If logs are stored on the root volume:

- Disk will eventually become full
- Application may crash

Solution:

Attach a new EBS volume and store MySQL logs separately.

The screenshot displays the AWS Management Console interface for launching an EC2 instance. The 'Launch an instance' page is active, showing the 'Name and tags' section with the instance name 'my-sql-db'. The 'Application and OS Images' section is expanded, showing a search bar and a grid of operating system images. The 'Summary' section on the right provides a overview of the configuration: 1 instance, Canonical, Ubuntu, 24.04, amd64 image, t5.micro instance type, New security group, and 1 volume(s) - 8 GiB storage. The 'Launch Instance' button is highlighted in orange.

EC2 > Instances > Launch an instance

Instance type

Instance type

m7i-flex.large

Family: m7i-flex 2 vCPU 8 GiB Memory Current generation: true Free tier eligible

On-Demand SUSE base pricing: 0.15705 USD per Hour On-Demand RHEL base pricing: 0.12955 USD per Hour

On-Demand Ubuntu Pro base pricing: 0.10425 USD per Hour On-Demand Linux base pricing: 0.10075 USD per Hour

On-Demand Windows base pricing: 0.18815 USD per Hour

Additional costs apply for AMIs with pre-installed software

[Compare instance types](#)

Key pair (login)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

peering-demo

[Create new key pair](#)

Network settings

Network

vpc-0b5fb1a6f36f6675a

Subnet

No preference (Default subnet in any availability zone)

Auto-assign public IP

Enable

Firewall (security groups)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group ☐ Select existing security group

We'll create a new security group called **launch-wizard-4** with the following rules:

☒ Allow SSH traffic from

Summary

Number of instances

1

Software Image (AMI)

Canonical, Ubuntu, 24.04, amd64...read more

ami-05d2d839d4f73aafb

Virtual server type (instance type)

m7i-flex.large

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

EC2 > Instances > Launch an instance

Availability Zone

No preference

[Enable additional zones](#)

Auto-assign public IP

Enable

Firewall (security groups)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group ☐ Select existing security group

Security group name - required

launch-wizard-4

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _-/@/.[!]+%&()*\$*

Description - required

launch-wizard-4 created 2026-04-04T07:54:38.092Z

Inbound Security Group Rules

Security group rule 1 (TCP, 22, 103.123.48.34/32)

Type ssh **Protocol** TCP **Port range** 22

Source type My IP **Name** **Description - optional** e.g. SSH for admin desktop

Security group rule 2 (TCP, 3306, 103.123.48.34/32)

Type MYSQL/Aurora **Protocol** TCP **Port range** 3306

Source type **Name** **Description - optional**

Summary

Number of instances

1

Software Image (AMI)

Canonical, Ubuntu, 24.04, amd64...read more

ami-05d2d839d4f73aafb

Virtual server type (instance type)

m7i-flex.large

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

After launch ssh into the ec2

```
sudo apt update
sudo apt install mysql-server -y
```

2. Start MySQL Service

```
sudo systemctl start mysql  
sudo systemctl enable mysql
```

Check Status

```
sudo systemctl status mysql
```

```
ubuntu@ip-172-31-47-193:~$ sudo systemctl status mysql  
● mysql.service - MySQL Community Server  
   Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)  
   Active: active (running) since Sat 2026-04-04 08:02:35 UTC; 53s ago  
     Main PID: 2307 (mysqld)  
    Status: "Server is operational"  
      Tasks: 38 (limit: 9279)  
     Memory: 363.6M (peak: 377.8M)  
        CPU: 827ms  
     CGroup: /system.slice/mysql.service  
             └─2307 /usr/sbin/mysqld  
  
Apr 04 08:02:35 ip-172-31-47-193 systemd[1]: Starting mysql.service - MySQL Community Server...  
Apr 04 08:02:35 ip-172-31-47-193 systemd[1]: Started mysql.service - MySQL Community Server.  
ubuntu@ip-172-31-47-193:~$
```

```
sudo mysql -u root -p OR sudo mysql  
Default password for the root will be: admin
```

```
ubuntu@ip-172-31-47-193:~$ sudo mysql -u root -p  
Enter password:  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 10  
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)  
  
Copyright (c) 2000, 2026, Oracle and/or its affiliates.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
mysql>
```

3. Create Database

```
CREATE DATABASE demo_db;
```

3. Use Database

```
USE demo_db;
```

Create Table (Interview + Practical Example)

Example: Employees Table

```
CREATE TABLE employees (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE,  
  salary DECIMAL(10,2),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

View Data

```
SELECT * FROM employees;
```

Insert Single Row (Basic)

```
INSERT INTO employees (name, email, salary)  
VALUES ('devops', 'devops@example.com', 45000.00);
```

Update Data

```
UPDATE employees  
SET salary = 70000  
WHERE name = 'Rahul';
```

Delete Data

```
DELETE FROM employees  
WHERE id = 1;
```

Insert Data

```
INSERT INTO employees (name, email, salary)  
VALUES  
( 'Gaurav', 'gaurav@example.com', 50000.00),  
( 'Rahul', 'rahul@example.com', 60000.00),  
( 'Priya', 'priya@example.com', 55000.00);
```

```
mysql> CREATE DATABASE demo db;  
Query OK, 1 row affected (0.01 sec)  
  
mysql> USE demo db;  
Database changed  
mysql>  
mysql> CREATE TABLE employees (  
-> id INT AUTO INCREMENT PRIMARY KEY,  
-> name VARCHAR(100) NOT NULL,  
-> email VARCHAR(100) UNIQUE,  
-> salary DECIMAL(10,2),  
-> created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
-> );  
Query OK, 0 rows affected (0.01 sec)  
  
mysql> SELECT * FROM employees;  
Empty set (0.01 sec)  
  
mysql> INSERT INTO employees (name, email, salary)  
-> VALUES  
-> ('Gaurav', 'gaurav@example.com', 50000.00),  
-> ('Rahul', 'rahul@example.com', 60000.00),  
-> ('Priya', 'priya@example.com', 55000.00);  
Query OK, 3 rows affected (0.00 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql>  
mysql>
```


View Data

SELECT * FROM employees;

```
mysql> SELECT * FROM employees;
+----+-----+-----+-----+-----+
| id | name  | email          | salary | created_at |
+----+-----+-----+-----+-----+
| 1  | Gaurav | gaurav@example.com | 50000.00 | 2026-04-04 08:08:58 |
| 2  | Rahul  | rahul@example.com  | 60000.00 | 2026-04-04 08:08:58 |
| 3  | Priya  | priya@example.com  | 55000.00 | 2026-04-04 08:08:58 |
+----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

Now we have the one ebs which contains the out sql data so we need to attach the new ebs volume to store the logs of mysql as when changes are made in db logs keep generating like read delta modify logs

2. Current Setup (Before Change)

Check current disk:

```
lsblk
df -hT
```

Observation

- Root volume (/) → 20GB
- MySQL logs currently stored on root disk
- No separate storage for logs

This is risky in production

So currently we have this setup

lsblk

```
ubuntu@ip-172-31-47-193:~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0        7:0      0   27.8M 1 loop /snap/amazon-ssm-agent/12322
loop1        7:1      0    74M 1 loop /snap/core22/2339
loop2        7:2      0   48.1M 1 loop /snap/snapd/25935
nvme0n1     259:0    0    20G 0 disk 
├─nvme0n1p1 259:1    0    19G 0 part /
├─nvme0n1p14 259:2   0     4M 0 part 
├─nvme0n1p15 259:3   0   106M 0 part /boot/efi
└─nvme0n1p16 259:4   0    913M 0 part /boot
ubuntu@ip-172-31-47-193:~$ df -hT
Filesystem      Type      Size  Used Avail Use% Mounted on
/dev/root       ext4       19G   2.6G   16G   14% /
tmpfs           tmpfs      3.8G     0   3.8G    0% /dev/shm
tmpfs           tmpfs      1.6G   1.1M   1.6G    1% /run
tmpfs           tmpfs      5.0M     0   5.0M    0% /run/lock
efivarfs        efivarfs   128K   3.6K  128K    3% /sys/firmware/efi/efivars
/dev/nvme0n1p16 ext4       881M   94M  726M   12% /boot
/dev/nvme0n1p15 vfat      105M    6.2M   99M    6% /boot/efi
tmpfs           tmpfs      778M    12K  778M    1% /run/user/1000
ubuntu@ip-172-31-47-193:~$
```

The screenshot displays the AWS Management Console interface for an EC2 instance. The left sidebar shows navigation options like Dashboard, AWS Global View, Events, Instances, Instance Types, Launch Templates, Spot Requests, Savings Plans, Reserved Instances, Dedicated Hosts, Capacity Reservations, Capacity Manager, Images, AMIs, AMI Catalog, Elastic Block Store, Volumes, Snapshots, Lifecycle Manager, Network & Security, Security Groups, Elastic IPs, Placement Groups, Key Pairs, and Network Interfaces.

The main content area shows the 'Instance summary for i-Uc4Zfaea283b2a943 (my-sql-db)'. The instance is in the 'Running' state. Key details include:

- Instance ID:** i-Uc4Zfaea283b2a943
- Public IPv4 address:** 3.110.127.115
- Private IPv4 addresses:** 172.31.47.193
- Public DNS:** ec2-3-110-127-115.ap-south-1.compute.amazonaws.com
- Instance type:** m7i-flex.large
- VPC ID:** vpc-0b5fb1a6f36f6675a
- Subnet ID:** subnet-0392d2f8084df4fe5
- Instance ARN:** arn:aws:ec2:ap-south-1:074095961179:instance/i-Uc4Zfaea283b2a943

The 'Storage' tab is selected, showing the following details:

- Root device details:** Root device name is /dev/sda1, and Root device type is EBS.
- Block devices:** A table lists the attached block devices.

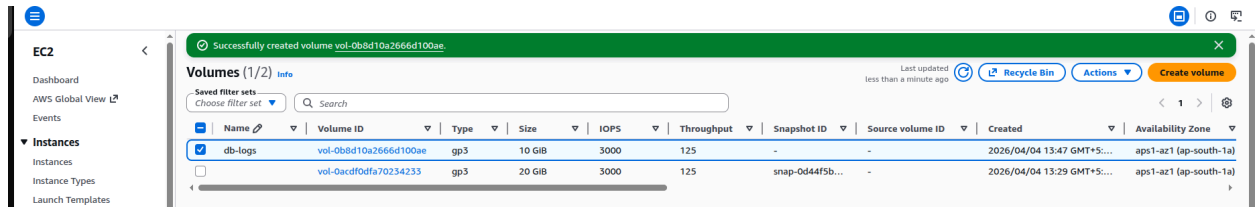
Volume ID	Device name	Volume size (GiB)	Volume State	Attachment status	EBS card index	Attachment time	Encrypted	KMS key ID
vol-0acdf0dfa70234233	/dev/sda1	20	In-use	Attached	0	2026/04/04 13:29 GMT+5:30	No	-

Create New EBS Volume

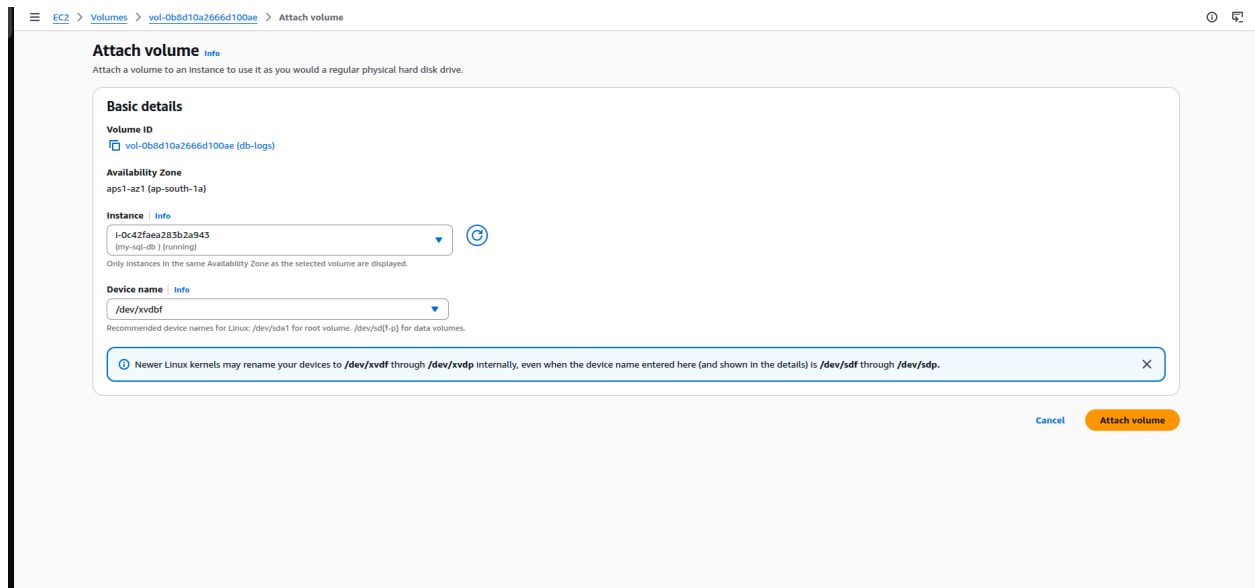
From AWS Console:

- Go to **EBS** → **Create Volume**
- Size: **10 GiB**
- AZ: **Same as EC2 instance (mandatory)**

So we have created new volume with 10Gib of size need to mount on the ec2 instance as we have just create it just like buying from market



Now it time to attach to the ec2 instance newly created instance



Now our instance has the 2 volumes

EC2 > Instances > i-0c42faea283b2a943

EC2

- Dashboard
- AWS Global View
- Events
- Instances**
 - Instances
 - Instance Types
 - Launch Templates
 - Spot Requests
 - Savings Plans
 - Reserved Instances
 - Dedicated Hosts
 - Capacity Reservations
 - Capacity Manager
- Images**
 - AMIs
 - AMI Catalog
- Elastic Block Store**
 - Volumes
 - Snapshots
 - Lifecycle Manager
- Network & Security**
 - Security Groups
 - Elastic IPs
 - Placement Groups
 - Key Pairs
 - Network Interfaces

Instance ID
i-0c42faea283b2a943

IPv6 address
-

Hostname type
IP name: ip-172-31-47-193.ap-south-1.compute.internal

Answer private resource DNS name
IPv4 (A)
5.110.127.115 [Public IP]

Auto-assigned IP address
5.110.127.115 [Public IP]

IAM role
-

IMDSv2
Required

Operator
-

Public IPv4 address
5.110.127.115 | [open address](#)

Instance state
Running

Private IP DNS name (IPv4 only)
ip-172-31-47-193.ap-south-1.compute.internal

Instance type
m7i-flex.large

VPC ID
vpc-0b5fb1a6f36f6675a

Subnet ID
subnet-0392d2f8084d4fe5

Instance ARN
arn:aws:ec2:ap-south-1:074095961179:instance/i-0c42faea283b2a943

Private IPv4 addresses
172.31.47.193

Public DNS
ec2-3-110-127-115.ap-south-1.compute.amazonaws.com | [open address](#)

Elastic IP addresses
-

AWS Compute Optimizer finding
[Opt-in to AWS Compute Optimizer for recommendations.](#) | [Learn more](#)

Auto Scaling Group name
-

Managed
false

Details | Status and alarms | Monitoring | Security | Networking | **Storage** | Tags

Root device details

Root device name
/dev/sda1

Root device type
EBS

EBS optimization
enabled

Block devices

Volume ID	Device name	Volume size (GiB)	Volume State	Attachment status	EBS card index	Attachment time	Encrypted	KMS key ID
vol-0acff0dffa70234233	/dev/sda1	20	In-use	Attached	0	2026/04/04 13:29 GMT+5:30	No	-
vol-0b8d10a2666d100ae	/dev/xvdbf	10	In-use	Attached	0	2026/04/04 13:51 GMT+5:30	No	-

lsblk

```
ubuntu@ip-172-31-47-193:~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0        7:0    0   27.8M  1 loop /snap/amazon-ssm-agent/12322
loop1        7:1    0    74M   1 loop /snap/core22/2339
loop2        7:2    0   48.1M  1 loop /snap/snapd/25935
nvme0n1      259:0    0    20G   0 disk
├─nvme0n1p1  259:1    0    19G   0 part /
├─nvme0n1p14 259:2    0     4M   0 part
├─nvme0n1p15 259:3    0   106M  0 part /boot/efi
├─nvme0n1p16 259:4    0   913M  0 part /boot
└─nvme1n1    259:5    0    10G   0 disk
ubuntu@ip-172-31-47-193:~$
```

And df -hT
Command show this

df -hT

```
ubuntu@ip-172-31-47-193:~$ df -hT
Filesystem      Type      Size  Used Avail Use% Mounted on
/dev/root        ext4       19G   2.6G   16G   14% /
tmpfs            tmpfs      3.8G     0   3.8G    0% /dev/shm
tmpfs            tmpfs      1.6G   1.1M   1.6G    1% /run
tmpfs            tmpfs      5.0M     0   5.0M    0% /run/lock
efivarfs         efivarfs   128K   3.6K   120K    3% /sys/firmware/efi/efivars
/dev/nvme0n1p16  ext4      881M   94M   726M   12% /boot
/dev/nvme0n1p15  vfat      105M   6.2M   99M    6% /boot/efi
tmpfs            tmpfs      778M   12K   778M    1% /run/user/1000
ubuntu@ip-172-31-47-193:~$
```

So we need to attach it

Now let create filesystem

Create File System

```
sudo mkfs -t ext4 /dev/nvme1n1
```

```
ubuntu@ip-172-31-47-193:~$ sudo mkfs -t ext4 /dev/nvme1n1
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 2621440 4k blocks and 655360 inodes
Filesystem UUID: 7cedeec3-3a19-4ade-82e1-dc6431a3bca7
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

ubuntu@ip-172-31-47-193:~$
```

Mount Volume

```
sudo mkdir /mysql-logs
sudo mount /dev/nvme1n1 /mysql-logs
```

Expected output:

```
df -hT
```

```

ubuntu@ip-172-31-47-193:~$ sudo mkdir /mysql-logs
ubuntu@ip-172-31-47-193:~$ sudo mount /dev/nvme1n1 /mysql-logs
ubuntu@ip-172-31-47-193:~$ df -hT

```

Filesystem	Type	Size	Used	Avail	Use%	Mounted on
/dev/root	ext4	19G	2.6G	16G	14%	/
tmpfs	tmpfs	3.8G	0	3.8G	0%	/dev/shm
tmpfs	tmpfs	1.6G	1.1M	1.6G	1%	/run
tmpfs	tmpfs	5.0M	0	5.0M	0%	/run/lock
efivarfs	efivarfs	128K	3.6K	120K	3%	/sys/firmware/efi/efivars
/dev/nvme0n1p16	ext4	881M	94M	726M	12%	/boot
/dev/nvme0n1p15	vfat	105M	6.2M	99M	6%	/boot/efi
tmpfs	tmpfs	778M	12K	778M	1%	/run/user/1000
/dev/nvme1n1	ext4	9.8G	24K	9.3G	1%	/mysql-logs

```

ubuntu@ip-172-31-47-193:~$

```

Make It Persistent (CRITICAL)

If we skip this → volume disappears after reboot

Get UUID

sudo blkid

```

ubuntu@ip-172-31-47-193:~$ sudo blkid
/dev/nvme0n1p16: LABEL="BOOT" UUID="1bbab773-c98f-47ee-aa18-6277cc35d82f" BLOCK_SIZE="4096" TYPE="ext4" PARTUUID="57b8a7f7-8a89-4202-b5e0-c7223d3e62d"
/dev/nvme0n1p1: LABEL="cloudimg-rootfs" UUID="0df68d9a-e70c-413e-9a10-736ae3f00a7a" BLOCK_SIZE="4096" TYPE="ext4" PARTUUID="d43079ff-5308-4a29-8b13-319a73e19584"
/dev/nvme0n1p15: LABEL_FATBOOT="UEFI" LABEL="UEFI" UUID="AFA7-BC80" BLOCK_SIZE="512" TYPE="vfat" PARTUUID="4a299fcf-620a-4893-b749-8ac5eccc3f5e"
/dev/loop1: BLOCK_SIZE="131072" TYPE="squashfs"
/dev/nvme0n1p14: PARTUUID="fc06e51f-b621-4e08-a53a-426d4050156d"
/dev/loop2: BLOCK_SIZE="131072" TYPE="squashfs"
/dev/loop0: BLOCK_SIZE="131072" TYPE="squashfs"
/dev/nvme1n1: UUID="7cedeec3-3a19-4ade-82e1-dc6431a3bca7" BLOCK_SIZE="4096" TYPE="ext4"
ubuntu@ip-172-31-47-193:~$

```

Edit fstab

sudo vi /etc/fstab

Add:

UUID="262e4c23-f03b-4bf3-862d-5dcf60d90171" /mysql-logs ext4 defaults,nofail 0 2

Understand Current Log Location

Check current config:

sudo mysql -e "SHOW VARIABLES LIKE 'log_error';"

```
/var/log/mysql/error.log
```

Or:

```
sudo grep -i log /etc/mysql/mysql.conf.d/mysqld.cnf
```

Update fstab

```
sudo vi /etc/fstab
```

Add:

```
UUID=<your-uuid> /mysql-logs ext4 defaults,nofail 0 2
```

9. Step 7: Configure MySQL Logs Location

Check Current Log Location

```
sudo mysql -e "SHOW VARIABLES LIKE 'log_error';"
```

Create Log Directory

```
sudo mkdir -p /mysql-logs/mysql
```

Set Permissions (CRITICAL)

```
sudo chown -R mysql:mysql /mysql-logs  
sudo chmod 750 /mysql-logs/mysql
```

10. Step 8: Update MySQL Configuration

```
sudo vi /etc/mysql/mysql.conf.d/mysqld.cnf
```

Add / Modify

```
log_error = /mysql-logs/mysql/error.log  
slow_query_log = 1
```

```
slow_query_log_file = /mysql-logs/mysql/slow.log
general_log = 1
general_log_file = /mysql-logs/mysql/general.log
```

```
drwxrwxr-x 12 root syslog 4096 Apr  7 10:19 log
```

```
drwxr-x---  2 mysql  adm          4096 Apr  7 10:19 mysql
```

```
drwxr-xr-x 13 root root  4096 Apr  7 10:17 var
drwxrwxr-x 12 root syslog 4096 Apr  7 10:19 log
```

```
-rw-r----- 1 mysql adm 5543 Apr  7 10:55 error.log
```

```
sudo mkdir -p /mysql-logs/mysql && \
sudo chown -R mysql:mysql /mysql-logs && \
sudo find /mysql-logs -type d -exec chmod 750 {} \; && \
sudo find /mysql-logs -type f -exec chmod 640 {} \; && \
sudo touch /mysql-logs/mysql/error.log /mysql-logs/mysql/general.log /mysql-logs/mysql/slow.log
&& \
sudo chown mysql:mysql /mysql-logs/mysql/*.log && \
sudo chmod 640 /mysql-logs/mysql/*.log
```

After this critical step is this
Edit the apparmor file for the configuration of mysql

```
Sudo vi /etc/apparmor.d/usr.sbin.mysqld
```

Add these lines

```
/mysql-logs/ r,
/mysql-logs/** rwk,
```

This explicitly allows MySQL to:

- Read
- Write
- Lock files in that path


```
sudo systemctl restart apparmor  
sudo systemctl restart mysql
```

11. Step 9: Restart MySQL

```
sudo systemctl restart mysql
```

Verify Status

```
sudo systemctl status mysql
```

12. Step 10: Validate Logs

```
ls -l /mysql-logs/mysql
```

Expected Files

- error.log
 - slow.log
 - general.log
-

13. Final Validation

```
df -h
```

`/mysql-logs` usage should start increasing

14. Key Concepts to Explain to Students

Why Separate Logs?

- Logs grow continuously

- Prevent root disk from filling
 - Improve system stability
-

Architecture Insight

- Root volume → OS + Application
 - New EBS → Logs
-

Golden Line

“Never store logs on root volume in production.”

15. Common Mistakes

- Volume not in same AZ ❌
 - Forgot to mount ❌
 - Forgot `/etc/fstab` ❌
 - Wrong permissions ❌
 - MySQL fails to start ❌
-

16. Interview Answer (Production Ready)

To prevent disk exhaustion due to MySQL logs, I would attach a new EBS volume, mount it to a dedicated directory, update MySQL configuration to store logs there, and ensure persistence and correct permissions.

17. Real Production Insight

In real systems:

- Logs are forwarded to:
 - CloudWatch
 - ELK Stack
 - EBS acts as temporary buffer
-

This version is now:

- Clean
- Structured
- Production-ready
- Teach-friendly

Resize-EBS-Volumes

Production Scenario: Resizing EBS Volume Without Downtime (Ubuntu EC2)

1. Scenario Overview

In a production environment, your application requires more storage, and your EC2 instance's EBS volume is running out of space.

Since the application is critical, you **cannot stop the instance**, so we perform **live (online) EBS resizing**.

2. Step 1: Connect and Check Current State

After SSH into the EC2 instance, run:

lsblk

df -hT

```
ubuntu@ip-172-31-41-164:~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0        7:0      0  27.8M 1 loop /snap/amazon-ssm-agent/12322
loop1        7:1      0   74M 1 loop /snap/core22/2339
loop2        7:2      0  48.1M 1 loop /snap/snapd/25935
nvme0n1      259:0     0   20G 0 disk
├─nvme0n1p1  259:1     0   19G 0 part /
├─nvme0n1p14 259:2     0    4M 0 part
├─nvme0n1p15 259:3     0  106M 0 part /boot/efi
└─nvme0n1p16 259:4     0   913M 0 part /boot
ubuntu@ip-172-31-41-164:~$ df -hT
Filesystem      Type      Size  Used Avail Use% Mounted on
/dev/root       ext4       19G   2.1G   17G  12% /
tmpfs           tmpfs      456M     0  456M   0% /dev/shm
tmpfs           tmpfs      183M  888K  182M   1% /run
tmpfs           tmpfs       5.0M     0   5.0M   0% /run/lock
efivarfs        efivarfs   128K   3.6K  120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p16 ext4       881M   94M  726M  12% /boot
/dev/nvme0n1p15 vfat       105M   6.2M   99M   6% /boot/efi
tmpfs           tmpfs       92M    12K   92M   1% /run/user/1000
ubuntu@ip-172-31-41-164:~$
```

Current Output

lsblk

```
nvme0n1          20G disk
├─nvme0n1p1      19G part /
```

df -hT

```
/dev/root ext4 19G
```

Explanation:

- Disk (**nvme0n1**) → Total storage (EBS volume)
- Partition (**nvme0n1p1**) → Logical division of disk
- Filesystem (**ext4**) → How OS stores data
- Mount Point (**/**) → Root directory

At this stage, system has **~20GB usable storage**

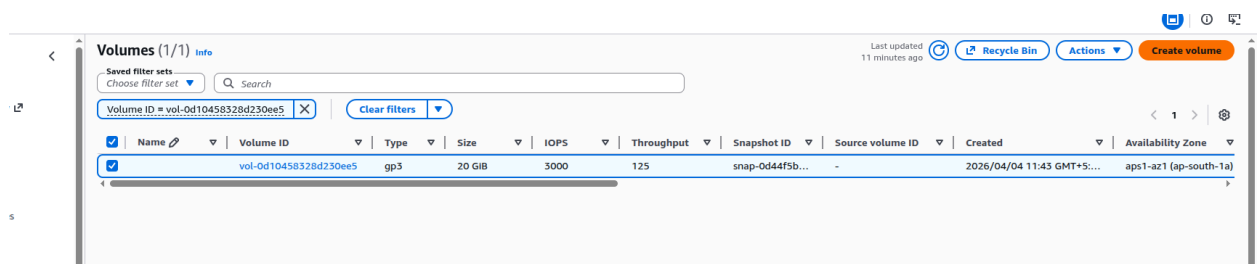
3. Step 2: Increase EBS Volume from AWS Console

Go to:

- EC2 → Volumes
 - Select volume (**vol-xxxx**)
 - Click **Modify Volume**
-

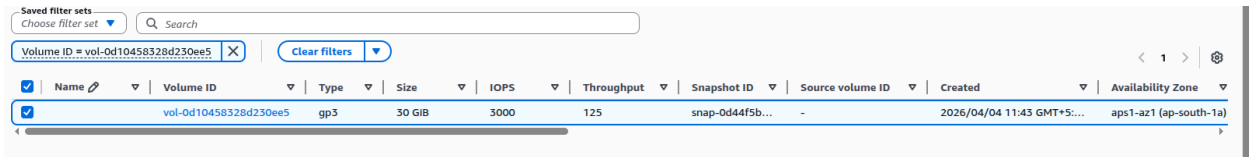
Before

- Size: **20 GB**



After

- Size: **30 GB**



The screenshot shows the AWS Management Console interface for an Amazon EBS volume. At the top, there is a 'Saved filter sets' section with a dropdown menu and a search bar. Below this, a filter is applied: 'Volume ID = vol-0d10458328d230ee5'. A 'Clear filters' button is also present. The main area displays a table of EBS volumes. The table has columns for Name, Volume ID, Type, Size, IOPS, Throughput, Snapshot ID, Source volume ID, Created, and Availability Zone. One volume is listed: 'vol-0d10458328d230ee5' of type 'gp3' with a size of '30 GiB'. The table is paginated, showing 1 of 1 items.

☒	Name	Volume ID	Type	Size	IOPS	Throughput	Snapshot ID	Source volume ID	Created	Availability Zone
☒		vol-0d10458328d230ee5	gp3	30 GiB	3000	125	snap-0d44f5b...	-	2026/04/04 11:43 GMT+5:...	aps1-az1 (ap-south-1a)

Important Concept

**AWS has increased the disk size
BUT OS is still using old partition size**

4. Step 3: Verify After Resize

Run:

lsblk

df -h


```

ubuntu@ip-172-31-41-164:~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0        7:0      0   27.8M  1 loop /snap/amazon-ssm-agent/12322
loop1        7:1      0    74M  1 loop /snap/core22/2339
loop2        7:2      0   48.1M  1 loop /snap/snapd/25935
nvme0n1      259:0    0    20G  0 disk
├─nvme0n1p1  259:1    0    19G  0 part /
├─nvme0n1p14 259:2    0     4M  0 part
├─nvme0n1p15 259:3    0   106M  0 part /boot/efi
└─nvme0n1p16 259:4    0   913M  0 part /boot
ubuntu@ip-172-31-41-164:~$ df -hT
Filesystem      Type      Size  Used Avail Use% Mounted on
/dev/root        ext4       19G   2.1G   17G  12% /
tmpfs            tmpfs      456M     0  456M   0% /dev/shm
tmpfs            tmpfs      183M   888K  182M   1% /run
tmpfs            tmpfs       5.0M     0   5.0M   0% /run/lock
efivarfs         efivarfs   128K   3.6K  120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p16 ext4       881M    94M  726M  12% /boot
/dev/nvme0n1p15 vfat      105M    6.2M   99M   6% /boot/efi
tmpfs            tmpfs      92M    12K   92M   1% /run/user/1000
ubuntu@ip-172-31-41-164:~$

```

Observation

Disk: 30G

Partition: still 19G

Filesystem: still 19G

Key Learning

Disk increased

Partition not increased

Filesystem not increased

5. Step 4: Expand Partition

`sudo growpart /dev/nvme0n1 1`

Explain to Students

- Disk = 30G
- Partition = still 19G

`growpart` expands partition to full disk

Expected Output

CHANGED: partition=1 ... new: size=...

```
ubuntu@ip-172-31-41-164:~$ sudo resize2fs /dev/nvme0n1p1
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/nvme0n1p1 is mounted on /; on-line resizing required
old_desc_blocks = 3, new_desc_blocks = 4
The filesystem on /dev/nvme0n1p1 is now 7601915 (4k) blocks long.
```

```
ubuntu@ip-172-31-41-164:~$
```

6. Step 5: Resize File System

Since filesystem is **ext4**, run:

`sudo resize2fs /dev/nvme0n1p1`

Explain to Students

- Partition is now bigger
- Filesystem still old size

This command updates filesystem to use full space

Expected Output

Filesystem on /dev/nvme0n1p1 is now ... blocks long

```
ubuntu@ip-172-31-41-164:~$ sudo resize2fs /dev/nvme0n1p1
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/nvme0n1p1 is mounted on /; on-line resizing required
old_desc_blocks = 3, new_desc_blocks = 4
The filesystem on /dev/nvme0n1p1 is now 7601915 (4k) blocks long.
```

7. Step 6: Final Verification

df -h

Final Output

/dev/root ~29G

```
ubuntu@ip-172-31-41-164:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        29G   2.1G   26G   8% /
tmpfs            456M   0     456M   0% /dev/shm
tmpfs            183M  892K   182M   1% /run
tmpfs            5.0M   0     5.0M   0% /run/lock
efivarfs         120K   3.0K   120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p16  881M   94M   726M  12% /boot
/dev/nvme0n1p15  105M   6.2M   99M    6% /boot/efi
tmpfs            92M   12K   92M    1% /run/user/1000
ubuntu@ip-172-31-41-164:~$
```

8. Final Result

- Storage increased from **20GB** → **30GB**
 - No downtime
 - Application continues running
-

9. Core Concept (Must Explain Clearly)

3 Layers of Storage

1. **Disk (EBS Volume)** → AWS level
 2. **Partition** → OS level
 3. **Filesystem** → Data level
-

Golden Line for Students

👉 “AWS gives you space, but Linux must be told to use it.”

10. Common Mistakes

- Only increasing volume in AWS
- Skipping **growpart**
- Skipping **resize2fs**

11. Interview Answer (Production Ready)

To resize an EBS volume in production without downtime, I would first increase the volume size from the AWS console, then expand the partition using `growpart`, and finally resize the filesystem using `resize2fs` or `xfs_growfs`.

12. Real Production Insight

- This is a **standard live operation**
 - No restart required
 - Used in real-world scaling scenarios
-

Scenario

Demo: Hosting MySQL on EC2 (Step-by-Step, Interview + Practical)

1. Architecture Thinking

For a proper setup:

- EC2 → Database server
- Security Group → Restrict access
- MySQL → Runs on port **3306**

Key rule (production mindset):

👉 Database should NOT be publicly accessible

2. Step 1: Launch EC2 Instance

Configuration

- AMI: Amazon Linux 2 / Ubuntu
 - Instance type: **t3.micro** (demo)
 - Key pair: create/download **.pem**
 - Storage: default (8–20 GB)
-

Security Group (CRITICAL PART)

Inbound Rules

- SSH → Port **22** → Your IP only
- MySQL → Port **3306**
 - **✗** NOT **0.0.0.0/0**
 - **✓** Only from:

- Your IP (for demo), OR
 - App server SG (production)
-

Outbound

- Allow all (default)
-

3. Step 2: Connect via SSH

```
chmod 400 mykey.pem  
ssh -i mykey.pem ec2-user@<public-ip>
```

For Ubuntu:

```
ssh -i mykey.pem ubuntu@<public-ip>
```

4. Step 3: Install MySQL Server

For Amazon Linux 2

```
sudo yum update -y  
sudo amazon-linux-extras enable mysql8.0  
sudo yum install mysql -y
```

For Ubuntu

```
sudo apt update  
sudo apt install mysql-server -y
```

5. Step 4: Start MySQL Service

```
sudo systemctl start mysqld  
sudo systemctl enable mysqld
```

Check Status

```
sudo systemctl status mysqld
```

6. Step 5: Secure MySQL (IMPORTANT)

```
sudo mysql_secure_installation
```

Set:

- Root password
 - Remove anonymous users
 - Disable remote root login
 - Remove test DB
-

7. Step 6: Access MySQL Locally

```
sudo mysql -u root -p
```

Password: admin

Create Database

```
CREATE DATABASE demo_db;
```

3. Use Database

```
USE demo_db;
```

Create Table (Interview + Practical Example)

Example: Employees Table

```
CREATE TABLE employees (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE,  
  salary DECIMAL(10,2),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

View Data

```
SELECT * FROM employees;
```

7. Insert Single Row (Basic)

```
INSERT INTO employees (name, email, salary)
VALUES ('Amit', 'amit@example.com', 45000.00);
```

8. Update Data

```
UPDATE employees
SET salary = 70000
WHERE name = 'Rahul';
```

9. Delete Data

```
DELETE FROM employees
WHERE id = 1;
```

10. Insert Data

```
INSERT INTO employees (name, email, salary)
VALUES
('Gaurav', 'gaurav@example.com', 50000.00),
('Rahul', 'rahul@example.com', 60000.00),
('Priya', 'priya@example.com', 55000.00);
```

10. Step 9: Test Connection

From local machine:

```
mysql -h <public-ip> -u demo_user -p
```

14. Interview Answer (Clean Version)

To host MySQL on EC2, I would launch an instance, configure security groups to allow SSH and controlled MySQL access, install and secure MySQL, create users and databases, and ensure the database is not publicly exposed in production by restricting access through security groups and private networking.

15. One-Line Strong Answer

You can host MySQL on EC2 by installing it on an instance and controlling access through security groups, but in production it should be secured and not exposed publicly.

Tab 12

Clean Demo Flow: MySQL Logs on New EBS

1. Step 1: Install MySQL (Keep it simple first)

```
sudo apt update
sudo apt install mysql-server -y
```

2. Start MySQL

```
sudo systemctl start mysql
sudo systemctl enable mysql
```

3. Login (IMPORTANT — don't confuse students)

```
sudo mysql
```

No password needed (auth_socket)

4. Create Demo Data (So logs have something to capture)

```
CREATE DATABASE demo_db;
USE demo_db;
```

```
CREATE TABLE test (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50)
);
```

```
INSERT INTO test (name) VALUES ('Gaurav');
SELECT * FROM test;
```

Explain:

- “Right now logs are going to default location (/var/log/mysql)”
-

NOW introduce the real problem

“Logs are growing → root disk will fill → production risk”

5. Step 2: Attach New EBS Volume

(Do from AWS Console)

Then verify:

```
lsblk
```

6. Create Filesystem

```
sudo mkfs -t ext4 /dev/nvme1n1
```

7. Mount Volume

```
sudo mkdir /mysql-logs  
sudo mount /dev/nvme1n1 /mysql-logs
```

8. Make Persistent

```
sudo blkid
```

```
sudo vi /etc/fstab
```

Add:

```
UUID="3de8164c-7b79-4fd0-a08c-035c5b083dfe" /mysql-logs ext4 defaults,nofail 0 2
```

9. Prepare Log Directory (VERY IMPORTANT)

```
sudo mkdir -p /mysql-logs/mysql
```

One clean command (students copy safely)

```
sudo mkdir -p /mysql-logs/mysql && \  
sudo chown -R mysql:mysql /mysql-logs && \  
sudo chmod 750 /mysql-logs/mysql && \  
sudo touch /mysql-logs/mysql/error.log /mysql-logs/mysql/general.log && \  
sudo chown mysql:mysql /mysql-logs/mysql/*.log && \  
sudo chmod 640 /mysql-logs/mysql/*.log
```

10. Update MySQL Config

```
sudo vi /etc/mysql/mysql.conf.d/mysqld.cnf
```

Add:

```
log_error = /mysql-logs/mysql/error.log  
general_log = 1  
general_log_file = /mysql-logs/mysql/general.log
```

11. Fix AppArmor (THIS IS THE REAL PRODUCTION STEP)

```
sudo vi /etc/apparmor.d/usr.sbin.mysqld
```

Add:

```
/mysql-logs/ r,  
/mysql-logs/** rwk,
```

```
sudo systemctl restart apparmor
```

12. Restart MySQL

```
sudo systemctl restart mysql
```

13. VERIFY

Generate Logs

```
sudo mysql
```

```
USE demo_db;  
INSERT INTO test (name) VALUES ('Rahul');  
SELECT * FROM test;
```

Check Logs

```
sudo tail -f /mysql-logs/mysql/general.log
```

will SEE:

```
INSERT INTO test ...  
SELECT * FROM test ...
```

This is the teaching moment

Say this:

“Now logs are no longer on root disk — they are safely stored on separate EBS”

14. Important Warning

Disable general log after demo:

```
general_log = 0
```

Then:

```
sudo systemctl restart mysql
```

Why?

- Performance impact
 - Disk growth
 - Not used in production
-

drwxrwxr-x 12 root syslog 4096 Apr 7 13:42 log

drwxr-x--- 2 mysql adm 4096 Apr 7 13:42 mysql

EC2-CLI

AWS EC2 CLI Cheat Sheet

0. Setup & Sanity Checks

```
aws configure
aws sts get-caller-identity
aws ec2 describe-regions
aws ec2 describe-availability-zones
```

If this fails, stop—credentials/region are wrong.

1. List / Describe EC2 Instances

List all instances

```
aws ec2 describe-instances
```

Clean output (interview-friendly)

```
aws ec2 describe-instances \
  --query
"Reservations[*].Instances[*].{ID:InstanceId,State:State.Name,Type:InstanceType,AZ:Placement
.AvailabilityZone,PublicIP:PublicIpAddress}" \
  --output table
```

Filter running instances

```
aws ec2 describe-instances \  
  --filters "Name=instance-state-name,Values=running"
```

2. Launch EC2 Instance

Basic launch

```
aws ec2 run-instances \  
  --image-id ami-xxxxxxx \  
  --count 1 \  
  --instance-type t2.micro \  
  --key-name my-key \  
  --security-group-ids sg-xxxx \  
  --subnet-id subnet-xxxx
```

With public IP (important)

```
aws ec2 run-instances \  
  --image-id ami-xxxxxxx \  
  --instance-type t2.micro \  
  --key-name my-key \  
  --network-interfaces  
  "AssociatePublicIpAddress=true,DeviceIndex=0,SubnetId=subnet-xxxx,Groups=sg-xxxx"
```

Tag instance at launch

```
aws ec2 run-instances \  
  --image-id ami-xxxxxxx \  
  --instance-type t2.micro \  
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=MyEC2}]'
```

3. Key Pair Management

Create key pair

```
aws ec2 create-key-pair \  
  --key-name my-key \  
  --query "KeyMaterial" \  
  --output text > my-key.pem
```

```
chmod 400 my-key.pem
```

List keys

```
aws ec2 describe-key-pairs
```

4. Security Group

Create SG

```
aws ec2 create-security-group \  
  --group-name my-sg \  
  --description "My SG" \  
  --vpc-id <VPC_ID>
```

Allow SSH

```
aws ec2 authorize-security-group-ingress \  
  --group-id sg-xxxx \  
  --protocol tcp \  
  --port 22 \  
  --cidr 0.0.0.0/0
```

Allow HTTP

```
aws ec2 authorize-security-group-ingress \  
  --group-id sg-xxxx \  
  --protocol tcp \  
  --port 80 \  
  --cidr 0.0.0.0/0
```

5. Instance Lifecycle

Start

```
aws ec2 start-instances --instance-ids i-xxxx
```

Stop

```
aws ec2 stop-instances --instance-ids i-xxxx
```

Reboot

```
aws ec2 reboot-instances --instance-ids i-xxxx
```

Terminate (delete)

```
aws ec2 terminate-instances --instance-ids i-xxxx
```

6. Get Instance Details (Very Important)

Public IP

```
aws ec2 describe-instances \  
  --instance-ids i-xxxx \  
  --query "Reservations[*].Instances[*].PublicIpAddress"
```

Private IP

```
aws ec2 describe-instances \  
  --instance-ids i-xxxx \  
  --query "Reservations[*].Instances[*].PrivateIpAddress"
```

7. AMI Handling

List AMIs

```
aws ec2 describe-images --owners amazon
```

Create AMI from instance

```
aws ec2 create-image \  
  --instance-id i-xxxx \  
  --name "MyAMI"
```

8. Volume (EBS) Operations

List volumes

```
aws ec2 describe-volumes
```

Create volume

```
aws ec2 create-volume \  
  --availability-zone ap-south-1a \  
  --size 10
```

Attach volume

```
aws ec2 attach-volume \  
  --volume-id vol-xxxx \  
  --instance-id i-xxxx \  
  --device /dev/xvdf
```

9. Tagging (Important in real infra)

Add tag

```
aws ec2 create-tags \  
  --resources i-xxxx \  
  --tags Key=Name,Value=MyServer
```

Filter by tag

```
aws ec2 describe-instances \  
  --filters "Name=tag:Name,Values=MyServer"
```

10. Advanced Query (Pro Level)

Get only instance IDs

```
aws ec2 describe-instances \  
  --query "Reservations[*].Instances[*].InstanceId" \  
  --output text
```

Get all running instance IDs (use in scripts)

```
aws ec2 describe-instances \  
  --filters "Name=instance-state-name,Values=running" \  
  --query "Reservations[*].Instances[*].InstanceId" \
```

--output text

11. Bulk Operations (Real DevOps Skill)

Stop all running instances

```
aws ec2 stop-instances \  
--instance-ids $(aws ec2 describe-instances \  
--filters "Name=instance-state-name,Values=running" \  
--query "Reservations[*].Instances[*].InstanceId" \  
--output text)
```

Terminate all instances (danger)

12. Common Mistakes

- Wrong AMI for region ❌
 - No public IP assigned ❌
 - SG not allowing traffic ❌
 - Wrong subnet (private but expecting internet) ❌
 - Key permission not set (chmod 400) ❌
-

13. Interview-Level Answer

If asked:

“How do you launch EC2 via CLI?”

They should say:

“Using `run-instances` with AMI ID, instance type, key pair, subnet, and security group, optionally attaching a public IP and tags.”

14. What Actually Makes Someone Strong

Not memorizing commands—but:

- Using `--query` effectively
 - Automating bulk operations
 - Understanding networking behind EC2
-

Load-Balancers

AWS Elastic Load Balancer (ELB)

1. What is an Elastic Load Balancer?

Elastic Load Balancing is a service that **automatically distributes incoming traffic across multiple targets** (EC2, containers, IPs) across multiple Availability Zones to ensure **high availability and fault tolerance**.

Golden Line

“ELB ensures no single server becomes a bottleneck or point of failure”

2. Why Do We Need a Load Balancer?

Without Load Balancer

- Traffic goes to single server
 - Server overload → crash
 - Downtime
-

With Load Balancer

- Traffic distributed
 - Failures handled automatically
 - High availability
-

3. Types of Load Balancers

1. Application Load Balancer (ALB)

- OSI Layer 7 (HTTP/HTTPS)
 - Content-based routing
-

2. Network Load Balancer (NLB)

- OSI Layer 4 (TCP/UDP)
 - Ultra high performance
-

3. Gateway Load Balancer (GWLB)

- OSI Layer 3
 - Used for firewalls / appliances
-

4. Classic Load Balancer (Legacy)

- Old generation
 - Not recommended Not Used by today
-

4. Core Components

1. Load Balancer

- Entry point for traffic
-

2. Listener

Listens on protocol + port

Example:

- HTTP : 80
- HTTPS : 443

Without listener → no traffic handled

3. Target Group

- Group of servers (EC2, IP, Lambda)
- Receives traffic

Each target group routes traffic to registered targets

4. Targets

- EC2 instances
 - Containers
 - IP addresses
-

5. Rules (ALB only)

- Define routing logic

Example:

- `/api` → backend server
 - `/images` → image server
-

5. How ELB Works (Flow)

Client → Load Balancer → Listener → Target Group → EC2

6. Health Checks (CRITICAL)

What it does

- Checks if instance is healthy
 - Sends traffic only to healthy targets
-

Example

- HTTP check `/health`
 - If fail → removed from rotation
-

7. Cross-Zone Load Balancing

OFF

- AZ A → only its instances
 - AZ B → only its instances
-

ON

- Traffic distributed across all AZs

Better distribution but more cross-AZ cost

8. Failure Handling (Important Concept)

DNS Failover

- Unhealthy AZ removed from DNS
 - Traffic goes to healthy AZ
-

Routing Failover

- Sends traffic to all targets (even unhealthy)
 - Prevents total outage
-

9. Listener Deep Concept (NLB)

Protocols Supported

- TCP
 - UDP
 - TLS
 - QUIC
-

TLS Listener

- Handles SSL termination
 - Offloads encryption from servers
-

Important

If using TCP → encryption passes to backend
If using TLS → LB decrypts traffic

10. Target Group Deep Concept

Target Types

- Instance
 - IP
 - Lambda
-

Important Behavior

- Can register same instance in multiple groups
 - Can scale dynamically
-

Routing Algorithms

- Round Robin
 - Least outstanding requests
 - Weighted
-

11. Stickiness (Session Persistence)

What it does

- Sends same user to same server
-

Use Case

- Login sessions
-

Problem

- Uneven load distribution
-

12. Real-Life Scenario

Web Application

- 2 EC2 instances
 - ALB distributes traffic
-

Flow

User → ALB → EC2-1 / EC2-2

13. Real-Life Scenario (Intermediate)

Microservices Routing

- `/api` → backend
 - `/static` → frontend
-

Uses ALB rules

14. Real-Life Scenario (Advanced)

Multi-AZ High Availability

- Instances in AZ A & AZ B
 - Load balancer distributes traffic
 - AZ failure → traffic shifts automatically
-

15. Production Architecture

Route53 → ALB → Auto Scaling Group → EC2

What happens

- Traffic comes via DNS
 - ALB distributes
 - Auto Scaling adds/removes instances
-

16. Key Advantages

- High availability
 - Auto scaling
 - Fault tolerance
 - SSL offloading
-

17. Common Mistakes

- ✗ Forgetting health checks
 - ✗ Not enabling cross-zone
 - ✗ Wrong security groups
 - ✗ No multi-AZ setup
-

18. Interview Questions

Q1: ALB vs NLB

- ALB → Layer 7 (HTTP routing)
 - NLB → Layer 4 (high performance)
-

Q2: What happens if instance fails?

Removed via health check

Q3: What is listener?

Entry point that listens on protocol + port

19. Final Production Insight

Load balancer is:

- Not optional
 - Mandatory for production
-

Without ELB:

- No scalability
 - No fault tolerance
-

Load Balancing Algorithms

Load Balancing Algorithms

1. What is a Load Balancing Algorithm? (Interview Definition)

A load balancing algorithm is a **method used by a load balancer to distribute incoming requests across multiple servers** to ensure optimal resource utilization, high availability, and performance.

Golden Line

👉 “Algorithm decides *which server gets the next request*”

2. Why Algorithms Matter

Without algorithm:

- Random traffic distribution
 - Some servers overloaded
 - Some servers idle
-

With algorithm:

- Balanced traffic
 - Better performance
 - High availability
-

3. Types of Load Balancing Algorithms

1. Static Algorithms

- 👉 Predefined rules
 - 👉 No real-time monitoring
-

2. Dynamic Algorithms

- 👉 Based on real-time server condition
 - 👉 More intelligent
-

4. Static Algorithms (Basic Level)

1. Round Robin

How it works

Request 1 → Server A
Request 2 → Server B
Request 3 → Server C
Request 4 → Server A (repeat)

Use Case

- Equal servers
 - Simple apps
-

Problem

👉 Ignores server load

2. Weighted Round Robin

How it works

- Server A (weight 2)
- Server B (weight 1)

👉 A gets double traffic

Use Case

- Different server capacity
-

Limitation

👉 Still static (no real-time adjustment)

3. Source IP Hash

How it works

- Same client IP → same server
-

Use Case

- Session persistence
-

Problem

👉 Uneven load possible

5. Dynamic Algorithms (Advanced Level)

1. Least Connections

How it works

👉 Send request to server with **least active connections**

Use Case

- High traffic systems
 - Long-running requests
-

Benefit

👉 Better load distribution

2. Least Response Time

How it works

👉 Sends request to fastest responding server

Use Case

- Low latency systems
-

Challenge

👉 Needs monitoring

3. Resource-Based

How it works

👉 Chooses server with:

- Lowest CPU
 - Lowest memory usage
-

Use Case

- Cloud / container systems
-

Complexity

👉 High

6. Static vs Dynamic (Interview Table)

Feature	Static	Dynamic
Decision	Fixed	Real-time
Performance	Basic	Optimized
Complexity	Low	High
Adaptability	No	Yes

7. How AWS ELB Uses This

👉 AWS hides complexity, but internally uses:

- Round Robin
 - Least Outstanding Requests (ALB)
 - Weighted routing
-

8. Real-Life Scenario

E-commerce Website

Problem

- High traffic during sale
-

Solution

- Load balancer distributes requests
-

Best Algorithm

- Least Connections
 - Weighted Round Robin
-
-

9. Production Decision Logic

When to Use What

Round Robin

- Small app
 - Equal servers
-

Weighted

- Mixed server sizes
-

Least Connections

- High traffic
 - Variable request time
-

Resource-Based

- Microservices / Kubernetes
-

10. Key Factors for Choosing Algorithm

- Traffic pattern
 - Server capacity
 - Latency requirements
 - Cost
-

11. Common Mistakes

- ✗ Always using Round Robin
 - ✗ Ignoring server capacity
 - ✗ Not monitoring performance
-

12. Interview Questions

Q1: Round Robin vs Least Connections

- Round Robin → sequential
 - Least Connections → real-time
-

Q2: Why dynamic algorithms are better?

- 👉 Adapt to real-time load
-

Q3: When to use IP hash?

- 👉 Session-based apps
-

13. Final Insight

- 👉 Load balancer ≠ intelligent by default
 - 👉 Algorithm = actual brain
-

EC2 Auto Scaling

AWS EC2 Auto Scaling

1. What is EC2 Auto Scaling?

Amazon EC2 Auto Scaling is a service that **automatically adds or removes EC2 instances based on demand** to maintain availability and optimize cost.

Golden Line

👉 “Auto Scaling ensures you always have the right number of instances — no more, no less”

2. Why Do We Need Auto Scaling?

Without Auto Scaling

- Fixed number of servers
 - Traffic spike → crash
 - Low traffic → wasted cost
-

With Auto Scaling

- Scale out (add instances)
- Scale in (remove instances)
- Cost optimized

3. Core Components

1. Launch Template

👉 Defines how instance should be created:

- AMI (OS)
 - Instance type (ex: t3.micro, t3.medium, t3x.large)
 - Key pair (public private key)
 - Security group
 - User data (bash script)
-

2. Auto Scaling Group (ASG)

👉 Logical group of EC2 instances

Capacity Settings

- **Min** → minimum instances
 - **Max** → maximum instances
 - **Desired** → current running instances
-

Example

Min = 2

Desired = 3

Max = 6

👉 Always maintains at least 2 instances

👉 Never exceeds 6

3. Scaling Policies

👉 Defines WHEN to scale

4. How Auto Scaling Works

User Traffic ↑ → CPU ↑ → **Scale Out** → Add Instances
User Traffic ↓ → CPU ↓ → **Scale In** → Remove Instances

5. Types of Scaling Policies

1. Target Tracking (Recommended)

Concept

👉 Like thermostat

Example:

Maintain CPU at 50%

- CPU 70% → add instance
 - CPU 30% → remove instance
-

2. Step Scaling

Example

CPU > 50% → +1 instance

CPU > 80% → +3 instances

3. Scheduled Scaling

Use Case

- Office apps
- School portals

Example:

9 AM → scale up

5 PM → scale down

4. Predictive Scaling (Advanced)

👉 Uses ML to predict traffic

6. Integration with Load Balancer

User → Load Balancer → Auto Scaling Group → EC2

Important Behavior

- New instance → automatically registered to LB
- Terminated instance → removed from LB

👉 This is automatic

7. Health Checks (CRITICAL)

Types

1. EC2 Health Check

- Checks instance status only
-

2. ELB Health Check (Recommended)

- Checks application health

👉 If app fails → instance replaced

8. Multi-AZ Deployment

👉 Instances distributed across multiple AZ

Benefit

- High availability
 - Fault tolerance
-

9. Advanced Features

1. Mixed Instance Policy

👉 Combine:

- On-Demand
- Spot Instances

👉 Reduce cost up to 90%

2. Lifecycle Hooks

Use Case

- Before start → install software
 - Before terminate → backup logs
-

3. Instance Refresh

👉 Rolling update of instances

10. Real-Life Scenario (Beginner)

Web Application

- 2 instances running
- Traffic increases

👉 ASG launches more instances

11. Real-Life Scenario (Intermediate)

E-commerce Sale

- Normal → 3 instances
- Sale → 10 instances

👉 Automatically scales

12. Real-Life Scenario (Advanced)

Production Architecture

Route53 → ALB → ASG → EC2 → RDS

Flow

- Traffic comes
- LB distributes

- ASG scales
-

13. Cost Optimization Insight

👉 Auto Scaling itself is FREE

👉 You pay for:

- EC2
 - Load balancer
 - CloudWatch
-

14. Common Mistakes

- ✗ Min = 1 (no HA)
 - ✗ Not using ELB health check
 - ✗ No scaling policy
 - ✗ Ignoring cost
-

15. Interview Questions

Q1: What happens if an instance fails?

👉 Auto Scaling replaces it automatically

Q2: Difference between Desired and Min?

- Desired → current target
 - Min → lower limit
-

Q3: Why use Target Tracking?

👉 Simple + automatic scaling

16. Final Production Insight

👉 Auto Scaling + Load Balancer = Mandatory combo

Without it:

- No scalability
- No fault tolerance

👉 Always think:

Traffic → LB → ASG → EC2 → Scaling Decision

EC2 Auto Scaling (Interview Ready)

One of the most common interview questions is:

"How do you scale an EC2 application when traffic increases?"

The answer is **Auto Scaling**, but interviewers expect you to explain **both Vertical Scaling and Horizontal Scaling**, when to use each, and their limitations.

What is Scaling?

Scaling means increasing or decreasing computing resources based on application demand.

Example:

Imagine you own a restaurant.

- 10 customers → 2 employees are enough.
- 200 customers → You need more resources.

You have two options:

1. Hire stronger employees (Vertical Scaling)
2. Hire more employees (Horizontal Scaling)

Cloud works exactly the same way.

1. Vertical Scaling (Scale Up / Scale Down)

Definition

Increasing or decreasing the resources of **one server**.

You replace the server with a bigger one.

Example:

t3.micro



t3.small



t3.medium



m5.large

Instead of adding more servers, you make the existing server more powerful.

Real-Life Example

Imagine a small grocery shop.

Initially:

- 1 cashier
- Small billing counter

During holidays:

Instead of opening another shop, you buy

- Faster computer
- Bigger billing machine
- More RAM
- Better processor

Same shop.

Just stronger hardware.

That is Vertical Scaling.

AWS Example

Current EC2

t3.micro

1 vCPU

1 GB RAM

Application becomes slow.

You stop the instance.

Change type

t3.large

2 vCPU

8 GB RAM

Start again.

Now application handles more requests.

Advantages

- ✓ Easy
 - ✓ No application redesign
 - ✓ Simple migration
-

Disadvantages

✗ Downtime

Instance must usually be stopped.

✗ Hardware limit

You cannot keep increasing forever.

Eventually the biggest EC2 instance is reached.

✗ Single Point of Failure

If that EC2 crashes

Entire application is down.

Applications Suitable

- Database servers
 - Development servers
 - Testing servers
 - Legacy applications
-

Interview Question

Can Auto Scaling Group perform Vertical Scaling?

Answer

No.

An Auto Scaling Group launches new instances.

Changing EC2 instance type is not Auto Scaling.

It is usually a manual activity or automation script.

2. Horizontal Scaling (Scale Out / Scale In)

Instead of increasing server size

Increase the number of servers.

Example

Before

EC2

After traffic increases

EC2

EC2

EC2

EC2

Traffic is distributed among all servers.

Real-Life Example

Imagine Domino's Pizza.

Too many customers arrive.

Do they make one chef work faster?

No.

They hire more chefs.

Every chef prepares pizzas.

Customers are divided among chefs.

Exactly how Load Balancer works.

AWS Architecture

Internet

|

Application Load Balancer

|

| | | |

EC2 EC2 EC2 EC2

Auto Scaling Group

ALB distributes requests.

Auto Scaling launches new EC2 instances.

Advantages

High Availability

No Single Point of Failure

Fault Tolerance

Easy scaling

Automatic scaling

No downtime

Disadvantages

Application must support distributed architecture.

Need Load Balancer.

Need shared storage or database.

Slightly more expensive.

Scale Out

Traffic increases

2 EC2

↓

4 EC2

↓

8 EC2

Scale In

Traffic decreases

8 EC2

↓

4 EC2

↓

2 EC2

AWS automatically terminates unnecessary EC2 instances.

Real AWS Example

Imagine Flipkart during a sale.

Morning

100 Users

2 EC2

Afternoon

10,000 Users

10 EC2

Midnight

200 Users

2 EC2

AWS automatically launches and terminates instances.

You pay only for what you use.

How Does AWS Know When to Launch New Instances?

Using **CloudWatch Metrics**.

Example:

If

CPU > 70%

for 5 minutes

Launch

+2 EC2

When

CPU < 25%

for 10 minutes

Terminate

-2 EC2

Components of EC2 Auto Scaling

1. Launch Template

Blueprint for creating EC2 instances.

Contains:

- AMI
- Instance Type

- Security Group
- IAM Role
- Key Pair
- User Data
- Storage Configuration

Think of it as a reusable template.

2. Auto Scaling Group (ASG)

Manages EC2 instances automatically.

Responsibilities:

- Launch instances
 - Remove instances
 - Replace unhealthy instances
 - Maintain desired capacity
 - Integrate with Load Balancer
-

3. Scaling Policy

Defines **when** to scale.

Example

CPU > 70%

Add 2 instances

4. CloudWatch

Monitors metrics.

Examples

- CPU
- Memory (custom metric)

- Network
 - Disk
 - Request Count
 - ALB Target Response Time
-

5. Application Load Balancer (ALB)

Distributes traffic across EC2 instances.

Without ALB

One server receives all traffic.

With ALB

Traffic is shared equally.

Auto Scaling Process

Users

↓

ALB

↓

CloudWatch monitors CPU

↓

CPU reaches 75%

↓

Scaling Policy triggered

↓

ASG launches 2 EC2 instances

↓

Launch Template used

↓

New EC2 registered with ALB

↓

Traffic distributed

↓

CPU drops

↓

ASG terminates extra EC2 instances

Interview Scenario

Your application suddenly receives one lakh users.

What happens?

Answer:

1. CPU usage increases.
 2. CloudWatch detects the high CPU utilization.
 3. The scaling policy is triggered.
 4. The Auto Scaling Group launches new EC2 instances using the Launch Template.
 5. The new instances automatically register with the Application Load Balancer.
 6. Incoming traffic is distributed across all healthy EC2 instances.
 7. When traffic decreases, the Auto Scaling Group terminates the extra instances according to the scale-in policy.
-

Common Interview Questions

Q1. What is the difference between Vertical and Horizontal Scaling?

Vertical Scaling	Horizontal Scaling
Increase server size	Increase server count
Scale Up / Down	Scale Out / In
Usually requires downtime	Usually no downtime
Limited by maximum instance size	Can scale to many instances
Single Point of Failure	High Availability and Fault Tolerance
Best for databases and legacy applications	Best for stateless web applications

Q2. Why do we use an Application Load Balancer with an Auto Scaling Group?

Because when new EC2 instances are launched, the Application Load Balancer automatically registers them and distributes incoming traffic only to healthy instances. This prevents any single EC2 instance from becoming overloaded and improves availability.

Q3. What happens if an EC2 instance becomes unhealthy?

The Auto Scaling Group detects the failed health check, terminates the unhealthy instance, and launches a replacement instance automatically using the Launch Template. This self-healing capability helps maintain the desired capacity.

Q4. What is Desired Capacity?

The number of EC2 instances the Auto Scaling Group tries to maintain at all times.

Example:

- Minimum Capacity: **2**
- Desired Capacity: **4**

- **Maximum Capacity: 10**

The ASG keeps **4 instances** running under normal conditions. It will never go below **2** or above **10** unless you change the configuration.

Q5. What is the difference between Desired, Minimum, and Maximum Capacity?

- **Minimum Capacity:** Lowest number of EC2 instances that must always be running.
 - **Desired Capacity:** Target number of EC2 instances the ASG maintains.
 - **Maximum Capacity:** Highest number of EC2 instances the ASG is allowed to launch.
-

Q6. Can an Auto Scaling Group work without an Application Load Balancer?

Yes. An ASG can launch, terminate, and replace EC2 instances without an ALB. However, for web applications, an ALB is typically used so that incoming traffic can be distributed across all healthy instances.

One-Line Interview Answer

Vertical Scaling means increasing the CPU, memory, or storage of a single EC2 instance, while **Horizontal Scaling** means adding or removing EC2 instances behind an **Application Load Balancer** using an **Auto Scaling Group** based on metrics from **CloudWatch**. Horizontal scaling is the preferred approach for production web applications because it provides high availability, fault tolerance, and automatic scaling with minimal downtime.

EC2 Advanced Concepts

EC2 Advanced Concepts

1. Termination Protection

What is it?

Termination Protection is a feature in EC2 that prevents an instance from being accidentally terminated.

Why is it important?

- Prevents accidental deletion of critical resources
 - Protects:
 - Production servers
 - Databases
 - Adds an extra safety layer
-

How it works

- When enabled:
 - Instance **cannot be terminated**
 - Even if someone tries via console or CLI → it fails
-

Key Insight

It does NOT prevent:

- Stop
- Reboot

Only protects against **termination**

Interview Line

Termination protection ensures that critical EC2 instances are not accidentally deleted, especially in production environments.

2. Stop vs Terminate (Very Important)

Stop Instance

- Instance is shut down
 - You can restart later
 - Data on EBS → preserved
 - Instance store → lost
-

Terminate Instance

- Instance is deleted permanently
 - Cannot be recovered
 - EBS (if configured) → deleted
-

Interview Trap

Many candidates confuse stop and terminate → basic rejection point

3. EC2 Instance Lifecycle

States

- Pending
 - Running
 - Stopping
 - Stopped
 - Terminated
-

Key Insight

- Billing:
 - Running → charged
 - Stopped → not charged (except storage)
-

4. Elastic IP (EIP)

What is it?

- Static public IPv4 address
-

Why is it needed?

- Default public IP changes on restart
 - EIP provides fixed IP
-

Important

- Charged if not used
-

Interview Line

Elastic IP ensures a static public IP for an EC2 instance, useful for consistent external access.

5. Placement Groups (Advanced but Important)

Types

Cluster

- Low latency
- Same hardware

Use Case:

- HPC
-

Spread

- Instances on different hardware

Use Case:

- High availability
-

Partition

- Logical partitions

Use Case:

- Big data (Hadoop, Spark)
-

6. EC2 Hibernation

What is it?

- Saves RAM state to disk
 - Instance resumes from same state
-

Why is it useful?

- Faster startup
 - Retain application state
-

Use Cases

- Long-running processes
 - Stateful apps
-

7. Security Groups vs NACL (Quick Advanced Insight)

Security Groups

- Instance-level
 - Stateful
-

NACL

- Subnet-level
 - Stateless
-

Interview Line

Security Groups control instance-level traffic, while NACLs provide subnet-level security control.

8. Instance Metadata (IMDS)

What is it?

- Endpoint inside EC2:

http://169.254.169.254

Used for

- Fetch:
 - IAM role credentials
 - Instance info
-

Critical Insight

- No need to store credentials manually
-

9. AMI (Amazon Machine Image)

What is it?

- Pre-configured template for EC2
-

Contains

- OS
 - Software
 - Configuration
-

Use Case

- Launch identical instances quickly

1. AMI (Amazon Machine Image)

Why it matters

👉 This is how you:

- Clone servers
 - Scale properly
 - Build golden images
-

Interview Expectation

“Create AMI → Auto Scaling uses it → consistent infra”

2. EC2 Networking Deep Dive

You touched VPC, but not enough:

You MUST know:

- ENI (Elastic Network Interface)
 - Public vs Private IP behavior
 - Elastic IP
 - Source/Destination check
-

👉 If this is weak → networking questions will expose you

3. Instance Metadata (IMDS)

Real-world use

curl http://169.254.169.254/latest/meta-data/

👉 Used for:

- IAM role credentials
 - Automation scripts
-

Interview Trap

👉 “How does EC2 get IAM credentials without storing keys?”

4. IAM Role for EC2 (CRITICAL)

Why important

👉 Secure access to:

- S3
 - DynamoDB
 - Secrets
-

Mistake people make

✗ Hardcoding access keys

5. EC2 Placement Groups

Types:

- Cluster → High performance
 - Spread → High availability
 - Partition → Big data
-

👉 Rare but strong differentiator in interviews

6. EC2 Storage Deep Dive (Missing Depth)

You covered EBS basics, but not:

- EBS types (gp2, gp3, io1, st1, sc1)
 - IOPS vs Throughput
 - EBS optimization
 - Instance store vs EBS
-

👉 This is asked in **mid-level interviews**

7. EC2 Monitoring

Must know:

- CloudWatch metrics
- CPU, Network, Disk
- Alarms → trigger scaling

8. EC2 Backup Strategy

Real-world:

- Snapshots
 - AMI backups
 - Automation
-
-

9. EC2 Security (Advanced)

You covered Security Groups basics — not enough.

You should know:

- SG vs NACL (deep comparison)
 - Bastion Host
 - Private instance access
 - Key rotation strategy
-
-

10. Real Production Scenario (MOST IMPORTANT)

👉 You must be able to explain:

User → Route53 → ALB → ASG → EC2 → RDS

If you can't explain:

- Where failure happens
- How scaling works
- How traffic flows

👉 You're not ready

AWS Production Architecture Lab

AWS Production Architecture Lab

Custom VPC + ALB + Target Group + Launch Template + Auto Scaling Group

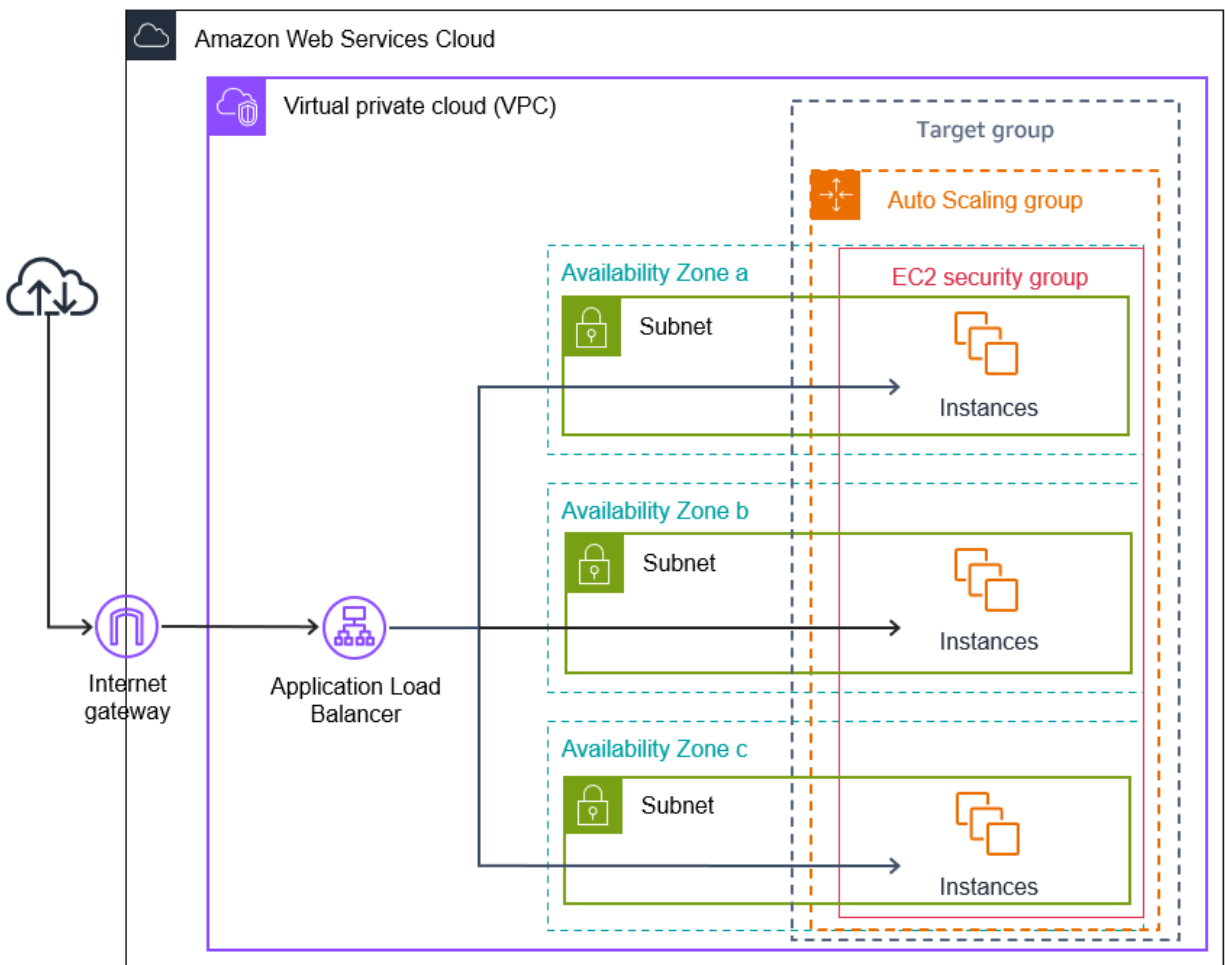
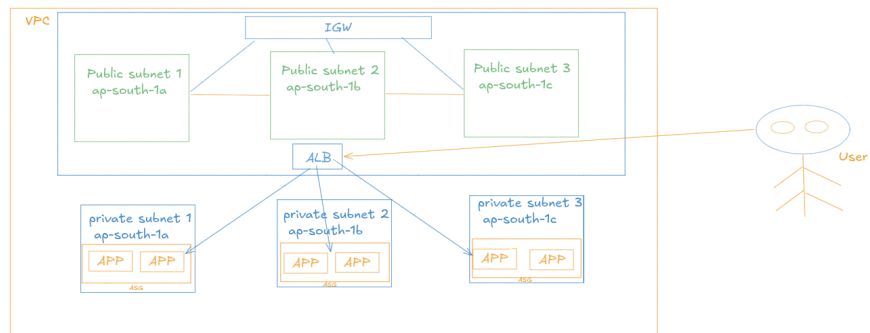
1. Architecture Overview

Internet → ALB (Public Subnet)

- Target Group
 - Auto Scaling Group (Private Subnets)
 - EC2 Instances
-

Key Design Decisions

- ALB in **public subnet**
 - EC2 in **private subnet**
 - Multi-AZ deployment
 - Auto Scaling enabled
 - Health checks enabled
-



2. Step 1: Create Custom VPC

VPC Configuration

- Name: **prod-vpc**
- CIDR: **10.0.0.0/16**

Create VPC [Info](#)

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.

VPC settings

Resources to create [Info](#)
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only ☐ VPC and more

Name tag - optional [Info](#)
Creates a tag with a key of 'Name' and a value that you specify.

prod-vpc

IPv4 CIDR block [Info](#)
☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR
10.0.0.0/16
CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)
☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

Tenancy [Info](#)
Default

VPC encryption control (\$) [Info](#)
Monitor mode provides visibility into encryption status without blocking traffic. Enforce mode prevents unencrypted traffic. [Additional charges apply](#)

☒ None ☐ Monitor mode ☐ Enforce mode

Monitor mode: See which resources in your VPC are unencrypted but allow the creation of unencrypted resources.

Enforce mode: Requires all resources, except exclusions, in your VPC to be encryption-capable and blocks creation of unencrypted resources.

CloudShell Feedback Console Mobile App © 2020, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Create Subnets (VERY IMPORTANT)

We create 6 subnets across 3 AZs

- 1) Name: public-1-ap-south-1a
subnet_Cidr: 10.0.1.0/24
Az: ap-south-1a

Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

public-1-ap-south-1a

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

Asia Pacific (Mumbai) / ap-south-1a

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

IPv4 subnet CIDR block

10.0.1.0/24 256 IPs

< > ^ v

▼ **Tags - optional**

Key	Value - optional	
Q Name	public-1-ap-south-1a	X Remove

[Add new tag](#)

You can add 49 more tags.

[Remove](#)

[Add new subnet](#)

[Cancel](#) [Create subnet](#)

2) Name: public-2-ap-south-1b
subnet_Cidr: 10.0.2.0/24
Az: ap-south-1b

Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

public-2-ap-south-1a

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

Asia Pacific (Mumbai) / ap-south-1b

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

IPv4 subnet CIDR block

10.0.2.0/24 256 IPs

< > ^ v

▼ **Tags - optional**

Key	Value - optional	
Q Name	public-2-ap-south-1a	X Remove

[Add new tag](#)

You can add 49 more tags.

[Remove](#)

[Add new subnet](#)

[Cancel](#) [Create subnet](#)

- 3) Name: public-3-ap-south-1c
subnet_Cidr: 10.0.3.0/24
Az: ap-south-1c

VPC > Subnets > Create subnet

IPv4 CIDR block
10.0.0.0/16

Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.
public-3-ap-south-1a
The name can be up to 256 characters long.

Availability Zone Info
Choose the zone in which your subnet will reside, or let Amazon choose one for you.
Asia Pacific (Mumbai) / ap-s1-az2 (ap-south-1c)

IPv4 VPC CIDR block Info
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.
10.0.0.0/16

IPv4 subnet CIDR block
10.0.3.0/24 256 IPs

Tags - optional

Key	Value - optional	
Name	public-3-ap-south-1a	Remove

[Add new tag](#)
You can add 49 more tags.

[Remove](#)

[Add new subnet](#)

CloudShell Feedback Console Mobile App

© 2020, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Private Subnets:

- 1) Name: private-1-ap-south-1a
subnet_Cidr: 10.0.11.0/24
Az: ap-south-1a

The screenshot shows the 'Create subnet' page in the AWS Management Console. The breadcrumb navigation is 'VPC > Subnets > Create subnet'. The page title is 'Subnet settings' with the instruction 'Specify the CIDR blocks and Availability Zone for the subnet.' Below this, it says 'Subnet 1 of 1'. The 'Subnet name' field is 'Private-Subnets1-ap-south-1a'. The 'Availability Zone' is 'Asia Pacific (Mumbai) / ap-south-1a'. The 'IPv4 VPC CIDR block' is '10.0.0.0/16'. The 'IPv4 subnet CIDR block' is '10.0.11.0/24' with a note '256 IPs'. There is a 'Tags - optional' section with one tag: 'Name' with value 'Private-Subnets1-ap-south-1a'. At the bottom right, there are 'Cancel' and 'Create subnet' buttons.

Subnet settings
Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.
Private-Subnets1-ap-south-1a
The name can be up to 256 characters long.

Availability Zone Info
Choose the zone in which your subnet will reside, or let Amazon choose one for you.
Asia Pacific (Mumbai) / ap-south-1a

IPv4 VPC CIDR block Info
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.
10.0.0.0/16

IPv4 subnet CIDR block
10.0.11.0/24 256 IPs

▼ **Tags - optional**

Key	Value - optional	
Name	Private-Subnets1-ap-south-1a	Remove

[Add new tag](#)
You can add 49 more tags.

[Remove](#)

[Cancel](#) [Create subnet](#)

- 2) Name: private-2-ap-south-1b
subnet_Cidr: 10.0.12.0/24
Az: ap-south-1b

The screenshot shows the 'Create subnet' page in the AWS Management Console. The breadcrumb navigation is 'VPC > Subnets > Create subnet'. The page title is 'Subnet settings' with the instruction 'Specify the CIDR blocks and Availability Zone for the subnet.' Below this, it says 'Subnet 2 of 1'. The 'Subnet name' field is 'private-2-ap-south-1b'. The 'Availability Zone' is 'Asia Pacific (Mumbai) / ap-south-1b'. The 'IPv4 VPC CIDR block' is '10.0.0.0/16'. The 'IPv4 subnet CIDR block' is '10.0.12.0/24' with a note '256 IPs'. There is a 'Tags - optional' section with one tag: 'Name' with value 'private-2-ap-south-1b'. At the bottom right, there are 'Cancel' and 'Create subnet' buttons.

Subnet settings
Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 2 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.
private-2-ap-south-1b
The name can be up to 256 characters long.

Availability Zone Info
Choose the zone in which your subnet will reside, or let Amazon choose one for you.
Asia Pacific (Mumbai) / ap-south-1b

IPv4 VPC CIDR block Info
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.
10.0.0.0/16

IPv4 subnet CIDR block
10.0.12.0/24 256 IPs

▼ **Tags - optional**

Key	Value - optional	
Name	private-2-ap-south-1b	Remove

[Add new tag](#)
You can add 49 more tags.

[Remove](#)

[Add new subnet](#)

[Cancel](#) [Create subnet](#)

- 3) Name: private-3-ap-south-1c
subnet_Cidr: 10.0.13.0/24
Az: ap-south-1c

VPC > Subnets > Create subnet

IPv4 CIDRs
10.0.0.0/16

Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.
private-3-ap-south-1c
The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.
Asia Pacific (Mumbai) / ap-s1-az2 (ap-south-1c)

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.
10.0.0.0/16

IPv4 subnet CIDR block
10.0.13.0/24 256 IPs

Tags - optional

Key	Value - optional	
Name	private-3-ap-south-1c	Remove

[Add new tag](#)
You can add 49 more tags.

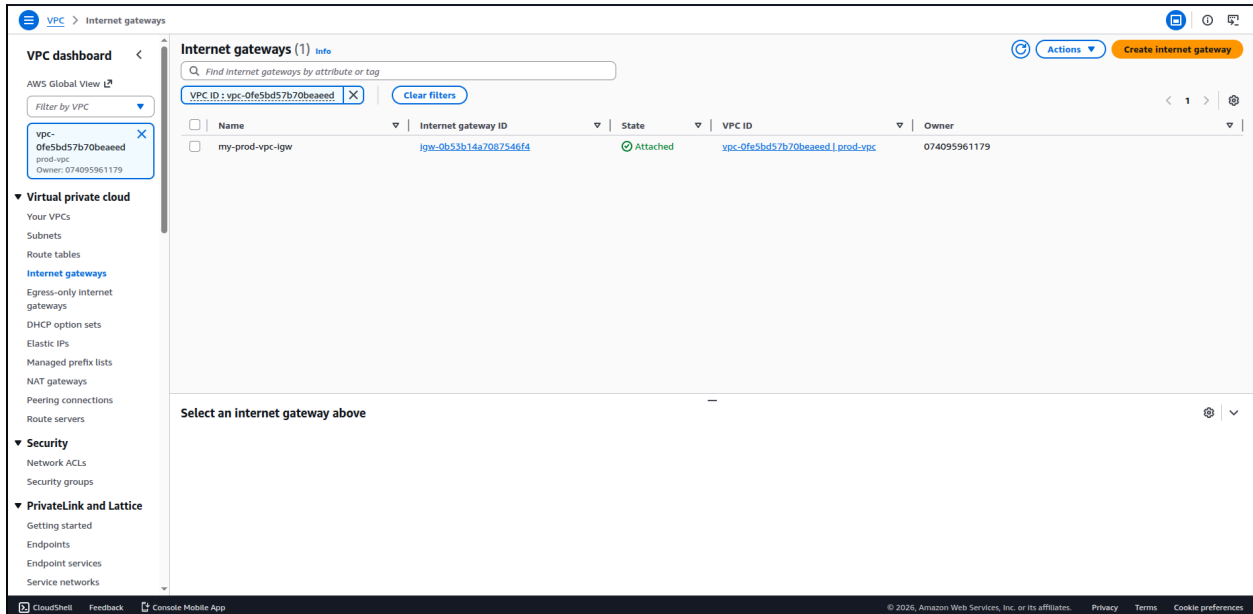
[Remove](#)

[Add new subnet](#)

CloudShell Feedback Console Mobile App © 2026 Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

STEP 3: Create Internet Gateway (IGW)

- Create IGW
- Attach to VPC



STEP 4: Route Table for Public Subnets

Create route table

Name: my-public-rt

Vpc: choose my-prod-vpc

VPCRoute tablesCreate route table

Create route table

info

A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.

Route table settings

Name - optional

Create a tag with a key of 'Name' and a value that you specify.

my-public-rt

VPC

The VPC to use for this route table.

vpc-0fe5bd57b70beaead (prod-vpc)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q NameX

Value - optional

Q my-public-rtXRemove

Add new tag

You can add 49 more tags.

CancelCreate route table

CloudShellFeedbackConsole Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

Add Route:

0.0.0.0/0 → Internet Gateway

VPCRoute tablesrtb-0b6cff940c4dc1590Edit routes

Edit routes

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable
Q 0.0.0.0/0X	Q localX	-	No	CreateRoute
	Internet Gateway			
	Q igw-0b53b14a7087546f4X			

Add route

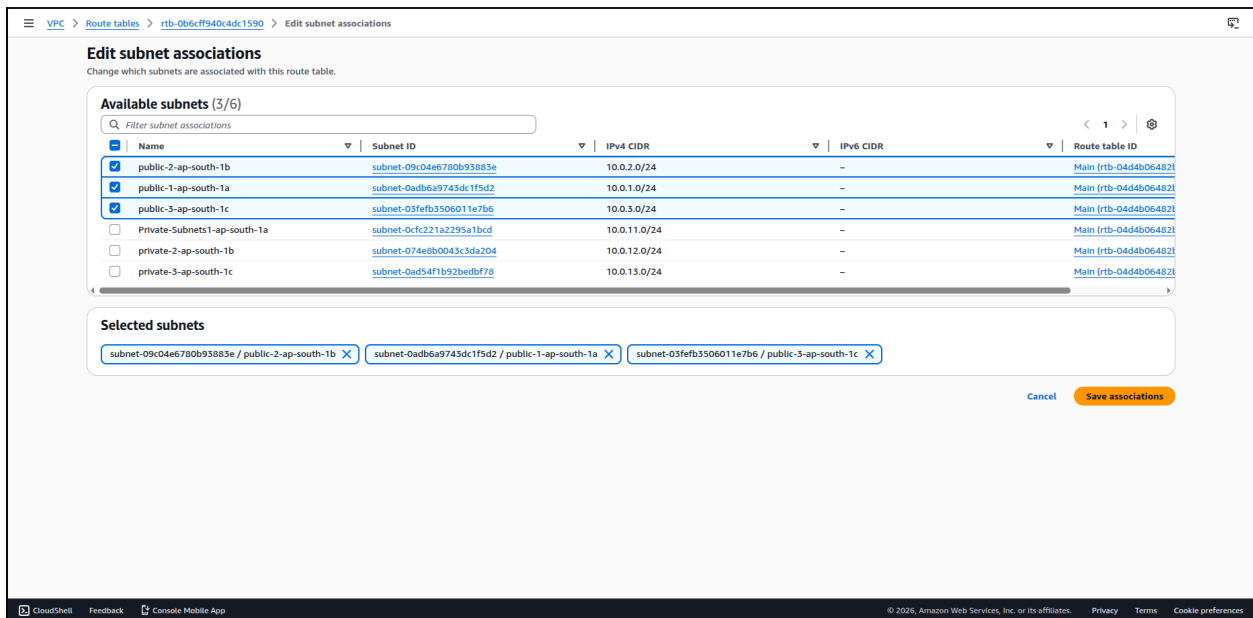
CancelPreviewSave changes

CloudShellFeedbackConsole Mobile App

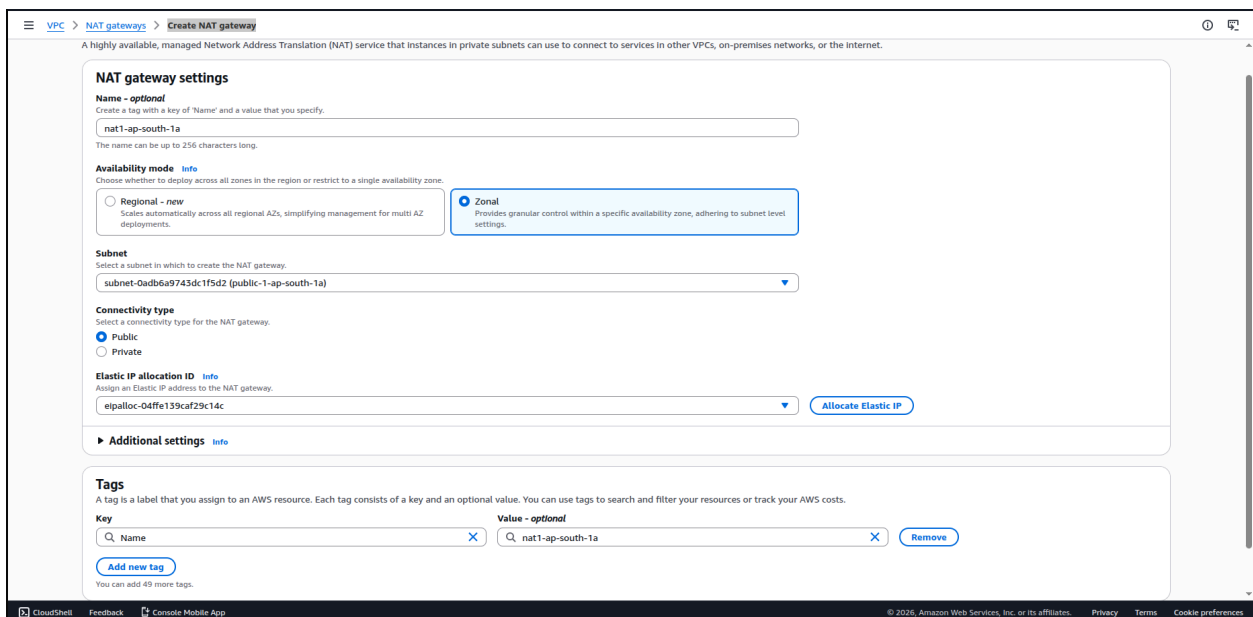
© 2026, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

Associate:

- public-1
- public-2
- public-3



STEP 5: Create NAT Gateways (CRITICAL)



VPC > NAT gateways > Create NAT gateway

Create NAT gateway

A highly available, managed Network Address Translation (NAT) service that instances in private subnets can use to connect to services in other VPCs, on-premises networks, or the internet.

NAT gateway settings

Name - optional

Create a tag with a key of 'Name' and a value that you specify.

nat1-ap-south-1b

The name can be up to 256 characters long.

Availability mode

Choose whether to deploy across all zones in the region or restrict to a single availability zone.

Regional - new

Scales automatically across all regional AZs, simplifying management for multi AZ deployments.

Zonal

Provides granular control within a specific availability zone, adhering to subnet level settings.

Subnet

Select a subnet in which to create the NAT gateway.

subnet-09c04e6790b93883e (public-2-ap-south-1b)

Connectivity type

Select a connectivity type for the NAT gateway.

Public

Private

Elastic IP allocation ID

Assign an Elastic IP address to the NAT gateway.

eipalloc-0df90b2dc5cd50a0c

Allocate Elastic IP

Additional settings

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name

X

Value - optional

Q nat1-ap-south-1b

X

Remove

Add new tag

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

VPC > NAT gateways > Create NAT gateway

Create NAT gateway

A highly available, managed Network Address Translation (NAT) service that instances in private subnets can use to connect to services in other VPCs, on-premises networks, or the internet.

NAT gateway settings

Name - optional

Create a tag with a key of 'Name' and a value that you specify.

nat1-ap-south-1c

The name can be up to 256 characters long.

Availability mode

Choose whether to deploy across all zones in the region or restrict to a single availability zone.

Regional - new

Scales automatically across all regional AZs, simplifying management for multi AZ deployments.

Zonal

Provides granular control within a specific availability zone, adhering to subnet level settings.

Subnet

Select a subnet in which to create the NAT gateway.

subnet-03fefb3506011e7b6 (public-3-ap-south-1c)

Connectivity type

Select a connectivity type for the NAT gateway.

Public

Private

Elastic IP allocation ID

Assign an Elastic IP address to the NAT gateway.

eipalloc-0a15c0923ef1c9bab

Allocate Elastic IP

Additional settings

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name

X

Value - optional

Q nat1-ap-south-1c

X

Remove

Add new tag

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

STEP 6: Route Tables for Private Subnets

- 1) **Name:** my-private-rt-ap-south-1a
Vpc: prod-vpc

The screenshot shows the 'Create route table' page in the AWS Management Console. The breadcrumb navigation is 'VPC > Route tables > Create route table'. The page title is 'Create route table' with an 'Info' link. A subtitle states: 'A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.'

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.
my-private-rt-ap-south-1a

VPC
The VPC to use for this route table.
vpc-0fe5bd57b70beaeed (prod-vpc)

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key
Name

Value - optional
my-private-rt-ap-south-1a

Buttons: Add new tag, Remove, Cancel, Create route table.

Footer: CloudShell, Feedback, Console Mobile App, © 2025, Amazon Web Services, Inc. or its affiliates. Privacy, Terms, Cookie preferences.

The screenshot shows the 'Edit routes' page in the AWS Management Console. The breadcrumb navigation is 'VPC > Route tables > rtb-0a1f98c6a87b89b7f > Edit routes'. The page title is 'Edit routes'.

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable
0.0.0.0/0	NAT Gateway	-	No	CreateRoute

Buttons: Add route, Remove, Cancel, Preview, Save changes.

Search results for NAT Gateway target:

- Use: "nat-00393a2cb5d1e071e"
- nat-0895afa852f26a68f (nat1-ap-south-1c)
- nat-06a62b73d3f9fab14 (nat1-ap-south-1b)
- nat-00393a2cb5d1e071e (nat1-ap-south-1a)

Selected route: nat-00393a2cb5d1e071e (nat1-ap-south-1a)

Footer: CloudShell, Feedback, Console Mobile App, © 2025, Amazon Web Services, Inc. or its affiliates. Privacy, Terms, Cookie preferences.

☰ VPC > Route tables > rtb-0a1f98c6a87b89b7f > Edit subnet associations

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (1/6)

Filter subnet associations

☒	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☐	public-2-ap-south-1b	subnet-09c04e6780b93883e	10.0.2.0/24	–	rtb-0b6cff940c4dc159c
☐	public-1-ap-south-1a	subnet-0adb6a9743dc1f5d2	10.0.1.0/24	–	rtb-0b6cff940c4dc159c
☐	public-3-ap-south-1c	subnet-03fe7b3506011e7b6	10.0.3.0/24	–	rtb-0b6cff940c4dc159c
☒	Private-Subnets1-ap-south-1a	subnet-0cf221a2295a1bcd	10.0.11.0/24	–	Main (rtb-04d4b06482f)
☐	private-2-ap-south-1b	subnet-074e8b0043c3da204	10.0.12.0/24	–	Main (rtb-04d4b06482f)
☐	private-3-ap-south-1c	subnet-0ad54f1b92bedbf78	10.0.13.0/24	–	Main (rtb-04d4b06482f)

Selected subnets

[subnet-0cf221a2295a1bcd / Private-Subnets1-ap-south-1a](#) X

Cancel Save associations

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

2) **Name:** my-private-rt-ap-south-1a
Vpc: prod-vpc

☰ VPC > Route tables > Create route table

Create route table

A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

VPC
The VPC to use for this route table.

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

X

Value - optional

X Remove

Add new tag

You can add 49 more tags.

Cancel Create route table

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

VPCRoute tablesrtb-09532feaded6f93adEdit routes

Destination

10.0.0.0/16

0.0.0.0/0

Add route

Target

local

local

NAT Gateway

nat-06a62b73d3f9fab14

Use: "nat-06a62b73d3f9fab14"

nat-0895afa052f26a68f (nat1-ap-south-1c)

nat-06a62b73d3f9fab14 (nat1-ap-south-1b)

nat-00393a2cb5d1e071e (nat1-ap-south-1a)

Status

Active

-

Propagated

No

No

Route Origin

CreateRouteTable

CreateRoute

Remove

Cancel

Preview

Save changes

VPCRoute tablesrtb-09532feaded6f93adEdit subnet associations

Available subnets (1/6)

Filter subnet associations

	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☐	public-2-ap-south-1b	subnet-09c04e6780b93883e	10.0.2.0/24	-	rtb-0b6cff940c4dc159c
☐	public-1-ap-south-1a	subnet-0adb6a9743dc1f5d2	10.0.1.0/24	-	rtb-0b6cff940c4dc159c
☐	public-3-ap-south-1c	subnet-03fefb3506011e7b6	10.0.3.0/24	-	rtb-0b6cff940c4dc159c
☐	Private-Subnets1-ap-south-1a	subnet-0cfc221a2295a1bcd	10.0.11.0/24	-	rtb-0a1f98c6a87b09b7
☒	private-2-ap-south-1b	subnet-074e8b0043c3da204	10.0.12.0/24	-	Main [rtb-04d4b06482f
☐	private-3-ap-south-1c	subnet-0ad54f1b92bed0f78	10.0.13.0/24	-	Main [rtb-04d4b06482f

Selected subnets

subnet-074e8b0043c3da204 / private-2-ap-south-1b

Cancel

Save associations

3) **Name:** my-private-rt-ap-south-1c
Vpc: prod-vpc

VPCRoute tablesCreate route table

Create route table

Route table settings

Name - optional

Create a tag with a key of 'Name' and a value that you specify.

Name: my-private-rt-ap-south-1c

VPC

The VPC to use for this route table.

vpc-0fe5bd57b70beaaed (prod-vpc)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name

Value - optional

Q Name: my-private-rt-ap-south-1c

Remove

Add new tag

You can add 49 more tags.

Cancel

Create route table

CloudShellFeedbackConsole Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

VPCRoute tablesrtb-0859a302fe4eb13f3Edit routes

Edit routes

Destination

10.0.0.0/16

Q 0.0.0.0/0

Add route

Target

local

Q local

NAT Gateway

Q nat-0895afa852f26a68f

Use: "nat-0895afa852f26a68f"

nat-0895afa852f26a68f (nat1-ap-south-1c)

nat-06a62b73d3f9fab14 (nat1-ap-south-1b)

nat-00393a2cb5d1e071e (nat1-ap-south-1a)

Status

Active

Propagated

No

No

Route Origin

CreateRouteTable

CreateRoute

Remove

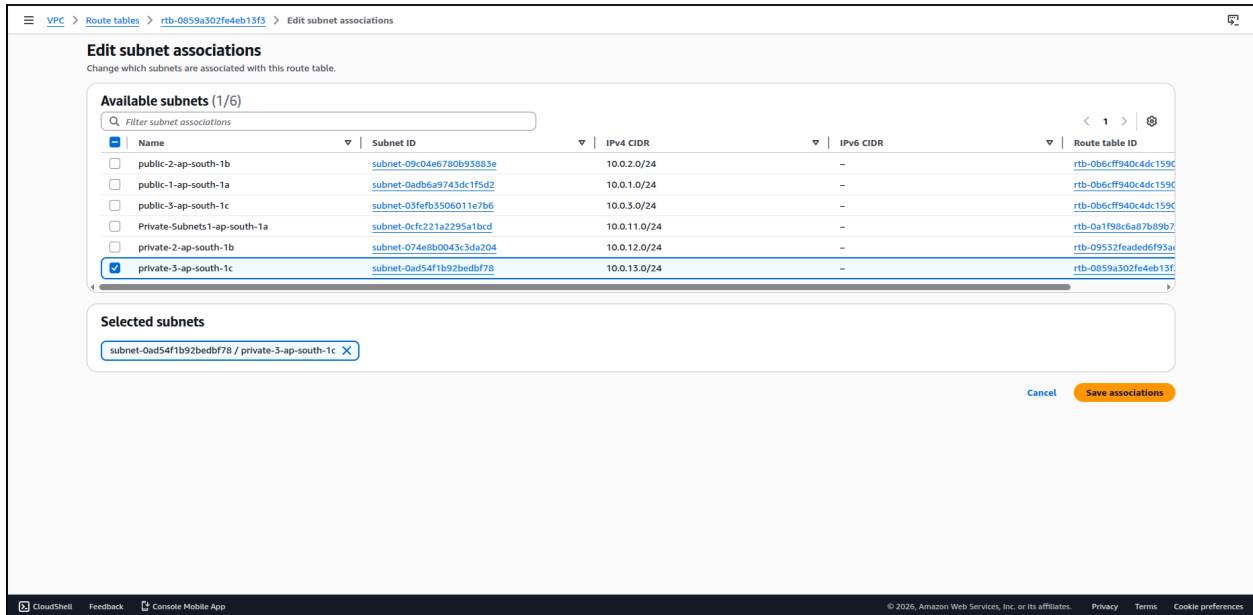
Cancel

Preview

Save changes

CloudShellFeedbackConsole Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences



STEP 7: Final Architecture (VERY IMPORTANT)

AZ-1:

Public-1 → IGW

Private-1 → NAT-1

AZ-2:

Public-2 → IGW

Private-2 → NAT-2

AZ-3:

Public-3 → IGW

Private-3 → NAT-3

STEP 8: Verification (DON'T SKIP)

Test 1: Public Subnet

- Launch EC2 → should get public IP
 - SSH works
-

Test 2: Private Subnet

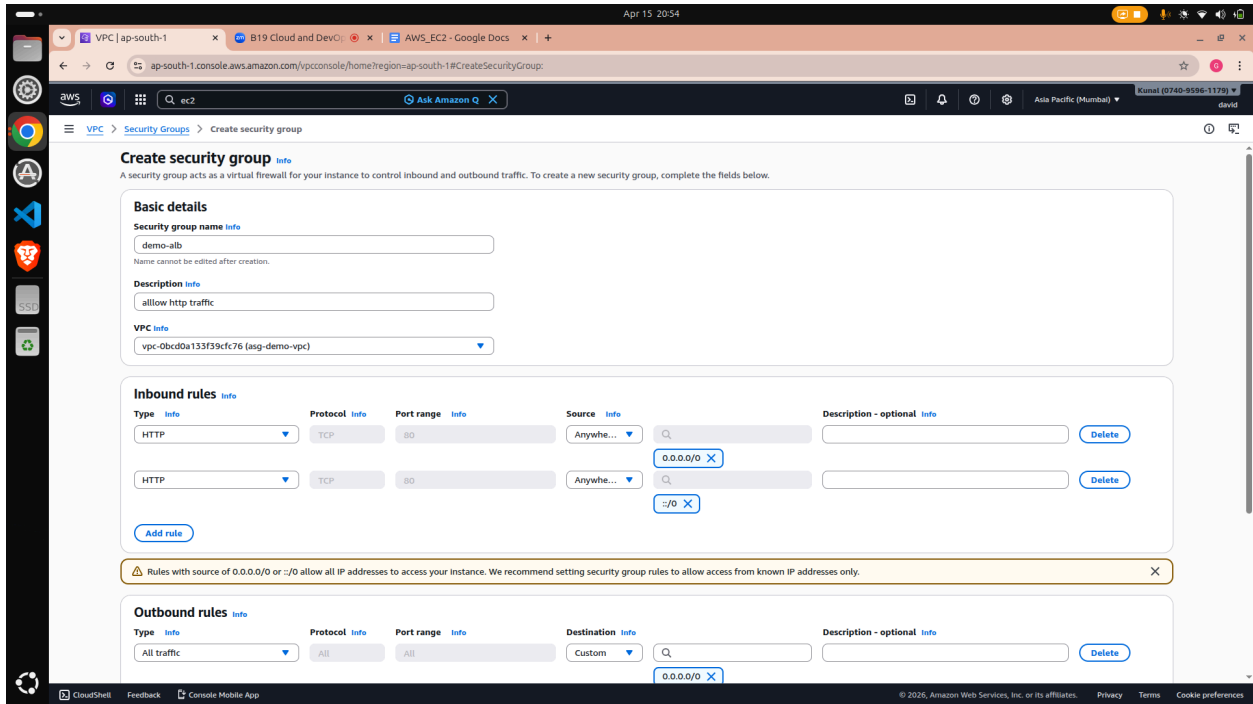
- Launch EC2 → no public IP
 - Use NAT → install packages works
-

Step 9: Security Groups (CRITICAL)

ALB Security Group

- Allow:

HTTP (80) → 0.0.0.0/0

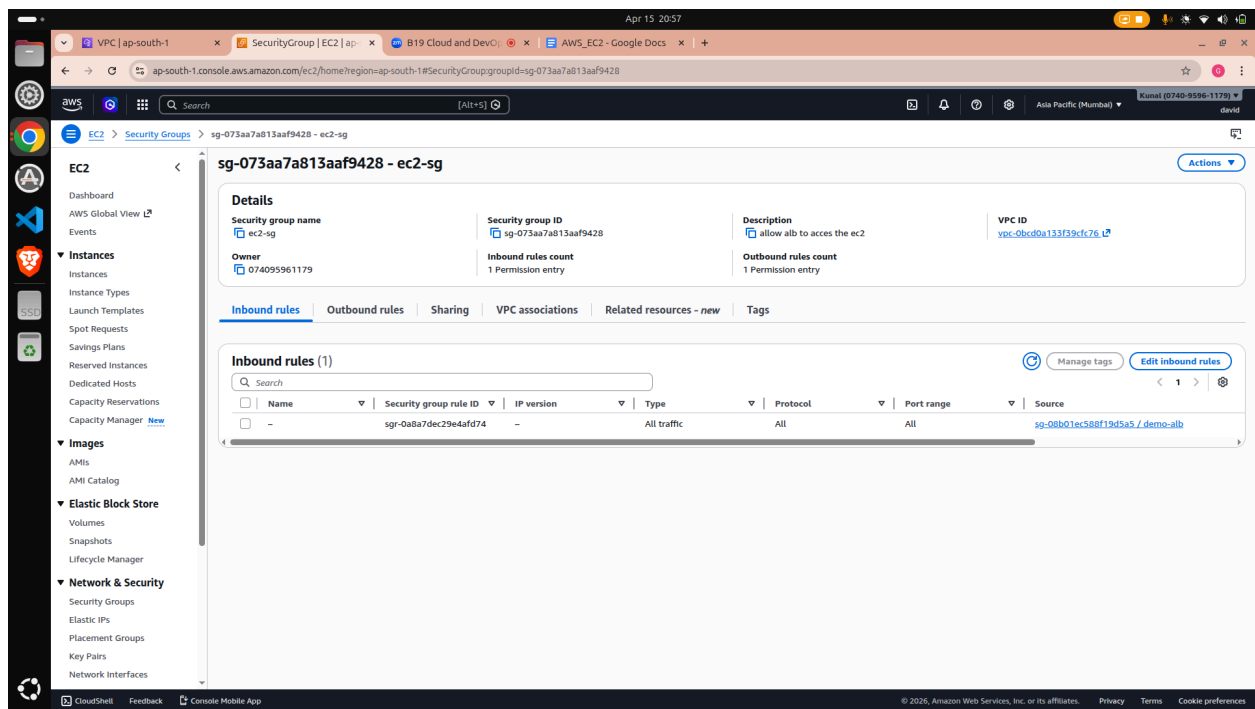


EC2 Security Group

- Allow:

HTTP (80) → ALB Security Group

👉 NOT from internet



Step 10: Launch Template

Configuration

- Name: **web-template**

Search results

Create launch template

Creating a launch template allows you to create a saved instance configuration that can be reused, shared and launched at a later time. Templates can have multiple versions.

Launch template name and description

Launch template name - *required*

Must be unique to this account. Max 128 chars. No spaces or special characters like '&', '"', '\', '@'.

Template version description

Max 255 chars

Auto Scaling guidance [Info](#)

Select this if you intend to use this template with EC2 Auto Scaling

☐ Provide guidance to help me set up a template that I can use with EC2 Auto Scaling

▶ Template tags

▶ Source template

Summary

Software Image (AMI)
Ubuntu Server 24.04 LTS (HVM),...[read more](#)
ami-05d2d839d4f73aafb

Virtual server type (instance type)
m7i-flex.large

Firewall (security group)
Ec2-sg

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Create launch template](#)

Launch template contents

Specify the details of your launch template below. Leaving a field blank will result in the field not being included in the launch template.

Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

[AMI from catalog](#) | [Recents](#) | [Quick Start](#)

Name
Ubuntu Server 24.04 LTS (HVM), SSD Volume Type

[Verified provider](#) [Free tier eligible](#)

[Browse more AMIs](#)

- AMI: Ubuntu / Amazon Linux

Search results

Launch template contents

Specify the details of your launch template below. Leaving a field blank will result in the field not being included in the launch template.

Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

[AMI from catalog](#) | [Recents](#) | [Quick Start](#)

Name
Ubuntu Server 24.04 LTS (HVM), SSD Volume Type

[Verified provider](#) [Free tier eligible](#)

Description
Ubuntu Server 24.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

Canonical, Ubuntu, 24.04, amd64 noble image

Image ID
ami-05d2d839d4f73aafb

Username ⓘ
ubuntu

Catalog	Published	Architecture	Virtualization	Root device type	ENA Enabled
Quick Start AMIs	2026-03-14T22:10:03.000Z	x86_64	hvm	ebs	Yes

Boot mode
uefi-preferred

[Browse more AMIs](#)
Including AMIs from AWS, Marketplace and the Community

Summary

Software Image (AMI)
Ubuntu Server 24.04 LTS (HVM),...[read more](#)
ami-05d2d839d4f73aafb

Virtual server type (instance type)
m7i-flex.large

Firewall (security group)
Ec2-sg

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Create launch template](#)

Instance type [Info](#) | [Get advice](#) [Advanced](#)

Instance type

- Instance type: t2.micro
- SG: EC2 SG
- Select and key pair just for the demo in production we dont use this

Search results

Metadata IPv6 endpoint

Info

Don't include in launch template

Metadata version

Info

Don't include in launch template

Metadata response hop limit

Info

Don't include in launch template

Allow tags in metadata

Info

Don't include in launch template

User data - optional

Info

Upload a file with your user data or enter it in the field.

Choose file

```
#!/bin/bash
apt update -y
apt install apache2 -y
systemctl start apache2
echo "Server: ${hostname}" > /var/www/html/index.html
```

☐ User data has already been base64 encoded

Summary

Software Image (AMI)

Ubuntu Server 24.04 LTS (HVM),...[read more](#)

ami-0562d839d4f73aaf6

Virtual server type (Instance type)

m7i-flex.large

Firewall (security group)

Ec2-sg

Storage (volumes)

1 volume(s) - 8 GiB

Cancel

Create launch template

CloudShell

Feedback

Console Mobile App

© 2024, Amazon Web Services, Inc. or its affiliates

Privacy

Terms

Cookie preferences

7. Step 6: Target Group

Configuration

- Type: Instance
- Protocol: HTTP
- Port: 80

The screenshot shows the 'Create target group' page in the AWS Management Console. The breadcrumb navigation at the top reads 'EC2 > Target groups > Create target group'. On the left, a progress bar shows four steps: 'Step 1: Create target group' (selected), 'Step 2 - recommended: Register targets', 'Step 3', and 'Review and create'. The main content area is titled 'Create target group' with a subtitle: 'A target group can be made up of one or more targets. Your load balancer routes requests to the targets in a target group and performs health checks on the targets.' Below this is a 'Settings - immutable' section with a warning: 'Choose a target type and the load balancer and listener will route traffic to your target. These settings can't be modified after target group creation.' The 'Target type' section asks to 'Indicate what resource type you want to target. Only the selected resource type can be registered to this target group.' It offers four options: 'Instances' (selected), 'IP addresses', 'Lambda function', and 'Application Load Balancer'. Each option has a description and a 'Suitable for' list of load balancer types (ALB, NLB, GWLB). The 'Instances' option is suitable for ALB, NLB, and GWLB. The 'IP addresses' option is suitable for ALB, NLB, and GWLB. The 'Lambda function' option is suitable for ALB. The 'Application Load Balancer' option is suitable for NLB. Below the target type section is the 'Target group name' field, which contains 'web-app-tg'. A note states: 'Name must be unique per Region per AWS account.' Below the name field is a note: 'Accepts: a-z, A-Z, 0-9, and hyphen (-). Can't begin or end with hyphen. 1-32 total characters; Count: 10/32'. The 'Protocol' section has a dropdown menu set to 'HTTP'. The 'Port' section has a text input field set to '80'. Below the port field is a note: 'Port number where targets receive traffic. Can be overridden for individual targets during registration.' The 'IP address type' section asks to 'Only targets with the indicated IP address type can be registered to this target group.' It offers two options: 'IPv4' (selected) and 'IPv6'. The 'IPv4' option has a note: 'Each instance has a default network interface (eth0) that is assigned the primary private IPv4 address. The instance's primary private IPv4 address is the one that will be applied to the target.' The 'IPv6' option has a note: 'Each instance you register must have an assigned primary IPv6 address. This is configured on the instance's default network interface (eth0). [Learn](#)'. At the bottom of the console, there is a footer with 'CloudShell', 'Feedback', 'Console Mobile App', and copyright information: '© 2020, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences'.

Health Check

Path: /

Interval: 30s

Healthy threshold: 2

Unhealthy threshold: 3

EC2 > Target groups > Create target group

VPC

Select the VPC with the instances that you want to include in the target group. Only VPCs that support the IP address type selected above are available in this list.

vpc-0ce34f53d01cd0c52 (prod-vpc-vpc)

10.0.0.0/16

Create VPC

Protocol version

☒ HTTP1

Send requests to targets using HTTP/1.1. Supported when the request protocol is HTTP/1.1 or HTTP/2.

☐ HTTP2

Send requests to targets using HTTP/2. Supported when the request protocol is HTTP/2 or gRPC, but gRPC-specific features are not available.

☐ gRPC

Send requests to targets using gRPC. Supported when the request protocol is gRPC.

Health checks

The associated load balancer periodically sends requests, per the settings below, to the registered targets to test their status.

Health check protocol

HTTP

Health check path

Use the default path of "/" to perform health checks on the root, or specify a custom path if preferred.

/

Up to 1024 characters allowed.

Advanced health check settings

Restore defaults

Health check port

The port the load balancer uses when performing health checks on targets. By default, the health check port is the same as the target group's traffic port. However, you can specify a different port as an override.

☒ Traffic port

☐ Override

Healthy threshold

The number of consecutive health check successes required before considering an unhealthy target healthy.

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

EC2 > Target groups > Create target group

Advanced health check settings

Restore defaults

Health check port

The port the load balancer uses when performing health checks on targets. By default, the health check port is the same as the target group's traffic port. However, you can specify a different port as an override.

☒ Traffic port

☐ Override

Healthy threshold

The number of consecutive health check successes required before considering an unhealthy target healthy.

2

2-10

Unhealthy threshold

The number of consecutive health check failures required before considering a target unhealthy.

5

2-10

Timeout

The amount of time, in seconds, during which no response means a failed health check.

5

seconds

2-120

Interval

The approximate amount of time between health checks of an individual target.

30

seconds

5-300

Success codes

The HTTP codes to use when checking for a successful response from a target. You can specify multiple values (for example, "200,202") or a range of values (for example, "200-299").

200

Target optimizer - optional

info

Use a target control port when the target has a strict concurrency limit.

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

8. Step 7: Application Load Balancer

Configuration

- Type: Application Load Balancer
- Scheme: Internet-facing

EC2 > Target groups > Create target group

▼ Advanced health check settings [Restore defaults](#)

Health check port
The port the load balancer uses when performing health checks on targets. By default, the health check port is the same as the target group's traffic port. However, you can specify a different port as an override.

☒ Traffic port
☐ Override

Healthy threshold
The number of consecutive health check successes required before considering an unhealthy target healthy.

2-10

Unhealthy threshold
The number of consecutive health check failures required before considering a target unhealthy.

2-10

Timeout
The amount of time, in seconds, during which no response means a failed health check.

seconds
2-120

Interval
The approximate amount of time between health checks of an individual target.

seconds
5-300

Success codes
The HTTP codes to use when checking for a successful response from a target. You can specify multiple values (for example, "200,202") or a range of values (for example, "200-299").

Target optimizer - optional [info](#)
Use a target control point when the target has a strict concurrency limit

CloudShell Feedback Console Mobile App © 2020, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

- Subnets: Public subnets
- SG: ALB SG

EC2 > Load balancers > Create Application Load Balancer

Use IPAM pool for public IPv4 addresses

The IPAM pool you choose will be the preferred source of public IPv4 addresses. If the pool is depleted IPv4 addresses will be assigned by AWS.

Availability Zones and subnets

Info

Select at least two Availability Zones and a subnet for each zone. A load balancer node will be placed in each selected zone and will automatically scale in response to traffic. The load balancer routes traffic to targets in the selected Availability Zones only.

ap-south-1a (apst1-az1)

Subnet

Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and Internet gateway (IGW) routes configured to allow traffic.

subnet-0f749d7662602208b (prod-vpc-subnet-public1-ap-south-1a)

10.0.0.0/20

Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (-)

ap-south-1b (apst1-az3)

Subnet

Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and Internet gateway (IGW) routes configured to allow traffic.

subnet-0514b819aac3adca6 (prod-vpc-subnet-public2-ap-south-1b)

10.0.16.0/20

Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (-)

Security groups

Info

A security group is a set of firewall rules that control the traffic to your load balancer. Listed security groups are in the same VPC as you selected above. Selected security groups must include at least 1 inbound rule and 1 outbound rule to allow traffic to your load balancer.

Security groups

Select up to 5 security groups

ALB-sg (sg-01c65fb6545f23920)

Inbound rule (1) Outbound rule (1)

Create security group

Listeners and routing

Info

A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.

Listener HTTP:80

Remove

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Listener

HTTP : 80 → Forward → Target Group

EC2 > Load balancers > Create Application Load Balancer

Listeners and routing

Info

A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.

Listener HTTP:80

Remove

Protocol

HTTP

Port

80

1-65535

Default action

Info

The default action is used if no other rules apply. Choose the default action for traffic on this listener.

Routing action

Forward to target groups

Redirect to URL

Return fixed response

Forward to target group

Info

Choose a target group and specify routing weight or create target group

Target group

web-app-tg

Target type: Instance, IPv4 | Target stickiness: Off

HTTP

Weight

1

0-999

Percent

100%

+ Add target group

You can add up to 4 more target groups.

Target group stickiness

Info

Enables the load balancer to bind a user's session to a specific target group. To use stickiness the client must support cookies. If you want to bind a user's session to a specific target, turn on the Target Group attribute Stickiness.

Turn on target group stickiness

Listener tags - optional

Consider adding tags to your listener. Tags enable you to categorize your AWS resources so you can more easily manage them.

Add listener tag

You can add up to 50 more tags.

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

9. Step 8: Auto Scaling Group

Configuration

- Launch Template: select template

EC2 > Auto Scaling groups > Create Auto Scaling group

Step 4: Configure group size and scaling

Name

Auto Scaling group name

web-app-asg

Must be unique to this account in the current Region and no more than 255 characters.

Launch template [Info](#)

For accounts created after May 31, 2023, the EC2 console only supports creating Auto Scaling groups with launch templates. Creating Auto Scaling groups with launch configurations is not recommended but still available via the CLI and API until December 31, 2023.

Launch template

Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.

web-template-launch

[Create a launch template](#)

Version

1

[Create a launch template version](#)

Description

this for the web application

Launch template

web-template-launch
lt-0a748ad97d4dd7723

Instance type

m7i-flex.large

AMI ID

ami-05d2d839d4f73aafb

Security groups

-

Request Spot Instances

No

Key pair name

-

Security group IDs

sg-03b9f7741767583ab

Additional details

Storage (volumes)

Date created

Thu Apr 14 2026 11:48:43 GMT+0530 (India Standard Time)

- Subnets: Private subnets

EC2 > Auto Scaling groups > Create Auto Scaling group

Step 7: Review

m7i-flex.large

Network [Info](#)

For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.

VPC

Choose the VPC that defines the virtual network for your Auto Scaling group.

vpc-0ce34f53d01cd0c52 (prod-vpc-vpc)

[Create a VPC](#)

Availability Zones and subnets

Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.

Select Availability Zones and subnets

aps1-az1 (ap-south-1a) | subnet-02b0ce04fb71b2d75 (prod-vpc-subnet-private1-ap-south-1a)

aps1-az3 (ap-south-1b) | subnet-02758de25072b9d74 (prod-vpc-subnet-private2-ap-south-1b)

[Create a subnet](#)

Availability Zone distribution - new

Auto Scaling automatically balances instances across Availability Zones. If launch failures occur in a zone, select a strategy.

Balanced best effort

If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.

Balanced only

If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.

Cancel Skip to review Previous Next

Attach Target Group

Mandatory step

The screenshot shows the 'Create Auto Scaling group' wizard in the AWS Management Console, specifically Step 3: 'Integrate with other services - optional'. The left sidebar shows the progress of the wizard, with Step 3 highlighted. The main content area is titled 'Integrate with other services - optional' and includes a description: 'Use a load balancer to distribute network traffic across multiple servers. Enable service-to-service communications with VPC Lattice. Shift resources away from impaired Availability Zones with zonal shift. You can also customize health check replacements and monitoring.'

Load balancing Info
Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.

Select Load balancing options

- ☐ No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.
- ☒ Attach to an existing load balancer
Choose from your existing load balancers.
- ☐ Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.

Attach to an existing load balancer

Select the load balancers to attach

- ☒ Choose from your load balancer target groups
This option allows you to attach Application, Network, or Gateway Load Balancers.
- ☐ Choose from Classic Load Balancers

Existing load balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.

Select target groups

web-app-tg | HTTP
Application Load Balancer: web-app-alb

VPC Lattice integration options Info
To improve networking capabilities and scalability, integrate your Auto Scaling group with VPC Lattice. VPC Lattice facilitates communications between AWS services and helps you connect and manage your applications across compute services in AWS.

Select VPC Lattice service to attach

- ☒ No VPC Lattice service
VPC Lattice will not manage your Auto Scaling group's network access and connectivity with other services.
- ☐ Attach to VPC Lattice service
Incoming requests associated with specified VPC Lattice target groups will be routed to your Auto Scaling group.

The footer of the console shows 'CloudShell', 'Feedback', 'Console Mobile App', and copyright information for Amazon Web Services, Inc. or its affiliates.

Capacity

Min = 2

Desired = 2

Max = 4

EC2

Auto Scaling groups

Create Auto Scaling group

Choose launch template

Step 2

Choose instance launch options

Step 3 - optional

Integrate with other services

Step 4 - optional

Configure group size and scaling

Step 5 - optional

Add notifications

Step 6 - optional

Add tags

Step 7

Review

Configure group size and scaling - optional

info

Define your group's desired capacity and scaling limits. You can optionally add automatic scaling to adjust the size of your group.

Group size

info

Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.

Desired capacity type

Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances)

Desired capacity

Specify your group size.

2

Scaling

info

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits

Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity

1

Equal or less than desired capacity

Max desired capacity

4

Equal or greater than desired capacity

Automatic scaling - optional

Choose whether to use a target tracking policy

info

You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.

☒ No scaling policies

Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.

☐ Target tracking scaling policy

Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.

Instance maintenance policy

info

Control your Auto Scaling group's availability during instance replacement events. This includes health checks, instance refreshes, maximum instance lifetime features and events that happen automatically to keep your group balanced, called rebalancing events.

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

10. Step 9: Scaling Policy

Target Tracking

CPU Utilization = 50%

EC2

Auto Scaling groups

Create Auto Scaling group

1

4

Equal or less than desired capacity

Equal or greater than desired capacity

Automatic scaling - optional

Choose whether to use a target tracking policy

info

You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.

☐ No scaling policies

Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.

☒ Target tracking scaling policy

Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.

Scaling policy name

Target Tracking Policy

Metric type

info

Monitored metric that determines if resource utilization is too low or high. If using EC2 metrics, consider enabling detailed monitoring for better scaling performance.

Average CPU utilization

Target value

50

Instance warmup

info

100 seconds

☐ Disable scale in to create only a scale-out policy

Instance maintenance policy

info

Control your Auto Scaling group's availability during instance replacement events. This includes health checks, instance refreshes, maximum instance lifetime features and events that happen automatically to keep your group balanced, called rebalancing events.

Choose a replacement behavior depending on your availability requirements

Minimized behavior

☒ No policy

For rebalancing events, new instances will launch before terminating others. For all other events, instances terminate and launch.

Prioritize availability

☐ Launch before terminating

Launch new instances and wait for them to be ready before terminating others. This allows you to go above your desired capacity for a short period of time.

Control costs

☐ Terminate and launch

Terminate and launch instances at the same time. This allows you to go below your desired capacity for a short period of time.

Flexibility

☐ Custom behavior

Set custom values for the minimum and maximum amount of available capacity. This shows you more flexibility in setting how big.

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

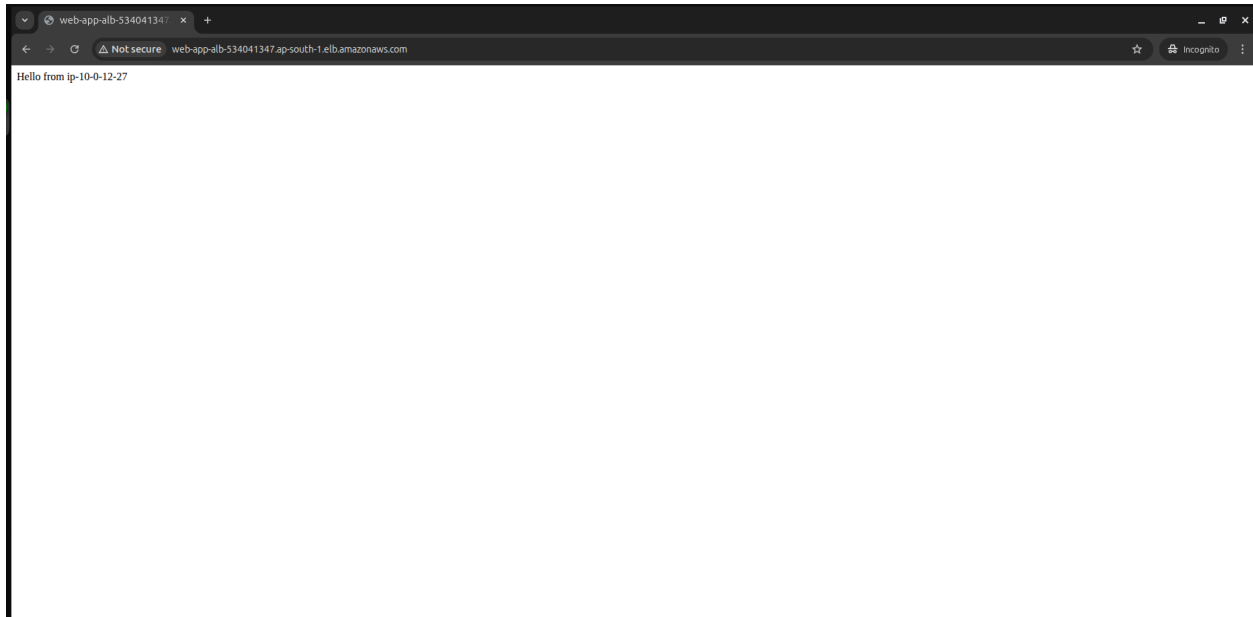
Privacy

Terms

Cookie preferences

11. Step 10: Testing

Access ALB DNS



Expected Result

- Refresh page → different server hostname

-
- Use target tracking ✓
 - Multi-AZ deployment ✓
 - Health Check: ELB ✓
 - Launch Template: Latest Version ✓
 - Monitoring: Enabled ✓
 - Warmup: 120–300 sec ✓
 - Tags: Added ✓

Monitoring

- Enable CloudWatch metrics ✓
-

16. Interview Explanation

“I designed a highly available architecture using a custom VPC with public and private subnets, deployed EC2 instances via a launch template in an Auto Scaling Group, and distributed traffic using an Application Load Balancer with health checks across multiple AZs.”

17. What Makes This Production Ready

- High availability (Multi-AZ)
 - Auto scaling
 - Secure network design
 - No direct internet exposure
 - Health-based routing
-

Interview-q/a

AWS (EC2 + VPC + ASG + ALB) –

SECTION 1: EC2 (Core + Advanced)

1. What is EC2?

EC2 is a virtual server in AWS that provides scalable compute without managing physical hardware.

2. EC2 vs VM?

EC2 is a VM, but AWS abstracts hardware, networking, scaling, and integration.

3. What happens when you stop vs terminate EC2?

- Stop → instance preserved, EBS retained
 - Terminate → instance deleted, data lost (unless EBS retained)
-

4. What is User Data?

Script executed at instance launch (bootstrapping).

5. Why did your user data fail earlier?

Because the private subnet had no NAT → no internet → `apt install` failed.

6. What is Instance Metadata?

Endpoint (`169.254.169.254`) providing instance info and IAM credentials.

7. Why not store AWS keys in EC2?

Use IAM roles → temporary credentials → secure.

8. What is ENI?

Elastic Network Interface → virtual NIC attached to EC2.

9. Difference: Public IP vs Elastic IP?

- Public IP → dynamic
 - Elastic IP → static
-

10. What is placement group?

Controls instance placement (cluster/spread/partition).

SECTION 2: VPC (Very Important)

11. What is VPC?

Logical isolated network in AWS.

12. Public vs Private subnet?

- Public → route to IGW
- Private → no direct IGW

13. Why private subnet cannot use IGW directly?

Because it lacks public IP + no direct route for return traffic.

14. What is NAT Gateway?

Allows private instances to access internet (outbound only).

15. Why NAT per AZ?

Avoid cross-AZ cost + ensure high availability.

16. What happens if NAT in AZ fails?

All private subnets using it lose internet.

17. Route Table role?

Defines traffic flow (where packets go).

18. What is NACL vs Security Group?

- SG → stateful
 - NACL → stateless
-

19. Why SG is stateful?

Return traffic automatically allowed.

20. What is CIDR?

IP range notation (e.g., 10.0.0.0/16).

SECTION 3: Load Balancer (ALB/NLB)

21. What is ALB?

Layer 7 load balancer (HTTP/HTTPS routing).

22. ALB vs NLB?

- ALB → Layer 7
 - NLB → Layer 4 (high performance)
-

23. What is Target Group?

Group of backend instances receiving traffic.

24. What is Listener?

Defines protocol/port (e.g., HTTP:80).

25. What is health check?

Checks if backend is alive and responding.

26. Why your target showed unhealthy?

No HTTP response → Apache not running.

27. What is 504 error?

Backend didn't respond in time.

28. Why use / in health check?

Independent endpoint, minimal dependency.

29. What is stickiness?

Same client → same backend.

30. Cross-zone load balancing?

Distributes traffic across AZs.

SECTION 4: Auto Scaling Group (ASG)

31. What is ASG?

Automatically manages EC2 instances.

32. Min, Max, Desired?

- Min → minimum instances
 - Desired → current
 - Max → limit
-

33. What is Launch Template?

Blueprint for EC2 creation.

34. Why versioning in LT?

Immutability + rollback.

35. Why ASG replaces instances?

Failed health check triggers replacement.

36. EC2 vs ELB health check?

- EC2 → OS-level
 - ELB → application-level
-

37. Why ELB health check better?

Detects real failures (app down, port closed).

38. What is scaling policy?

Defines when to scale.

39. Target tracking scaling?

Maintains metric (e.g., CPU 50%).

40. What is warmup period?

Delay before evaluating new instance.

SECTION 5: Real Scenario Questions

41. Why your architecture failed initially?

No NAT → app not installed → no response.

42. Why ALB showed unhealthy but EC2 healthy?

EC2 check passed, app check failed.

43. Why fixing one EC2 didn't solve issue?

ASG launches from broken Launch Template.

44. Why no public IP on EC2?

Security — only ALB exposed.

45. How EC2 in private subnet gets internet?

Via NAT Gateway.

46. Why not SSH directly?

No public IP → use SSM/Bastion.

47. What happens if SG misconfigured?

Traffic blocked → timeout.

48. Why multi-AZ deployment?

High availability.

49. Why tagging important?

Cost tracking + automation.

50. Full architecture explain

👉 Answer:

User → Route53 → ALB → Target Group → ASG → EC2 → Response

FINAL ADVICE (Important)

Most candidates:

- Memorize definitions
 - Fail in scenarios
-

👉 You must be able to explain:

- Why system failed
- Where traffic broke
- How to debug

AWS CloudWatch



AWS CloudWatch – Complete Production Documentation

1. Objective

Understand and implement:

- Metrics monitoring
 - Log aggregation
 - Alerting (Alarms)
 - Event-driven automation
 - Dashboards
-

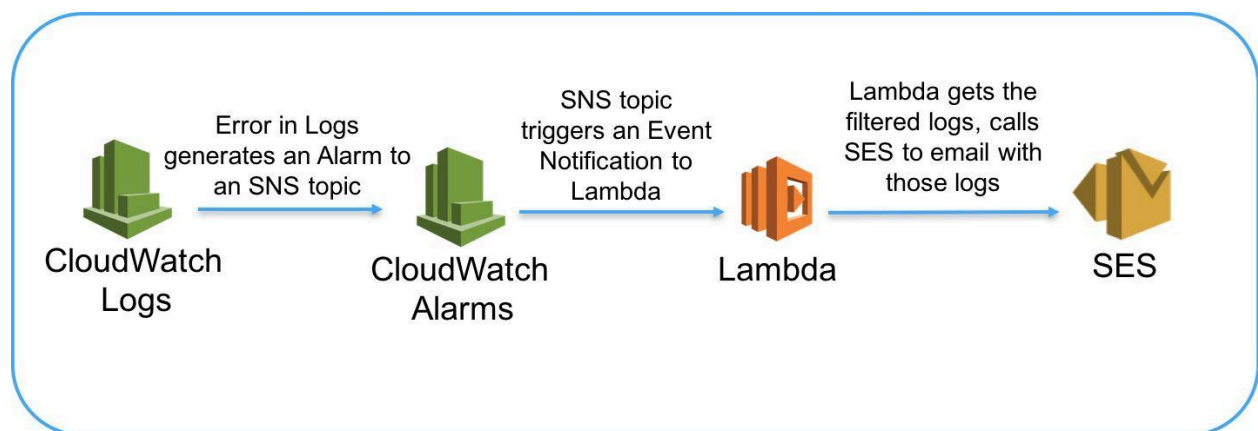
2. What is CloudWatch (Real Definition)

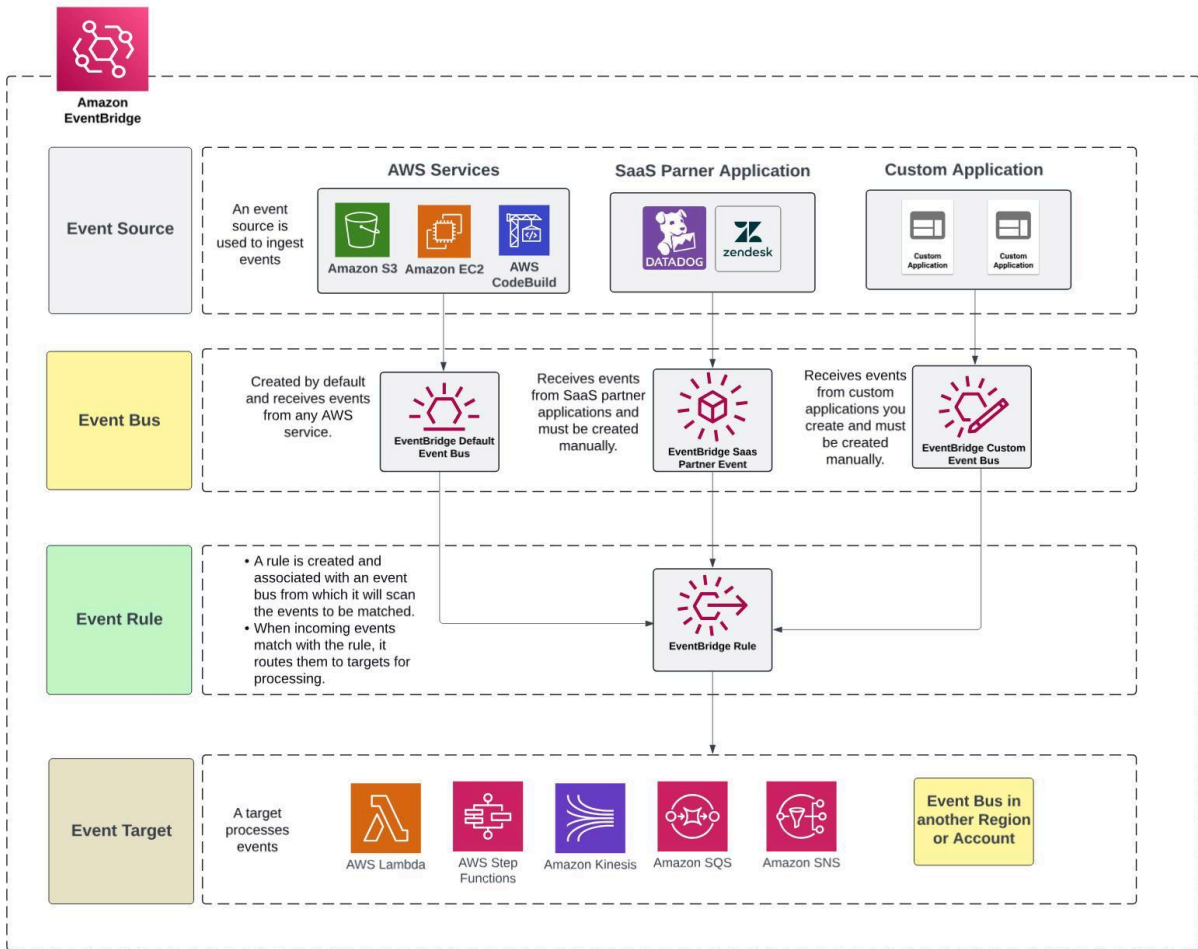
CloudWatch is a centralized observability service that collects, monitors, and analyzes metrics, logs, and events from AWS resources and applications.

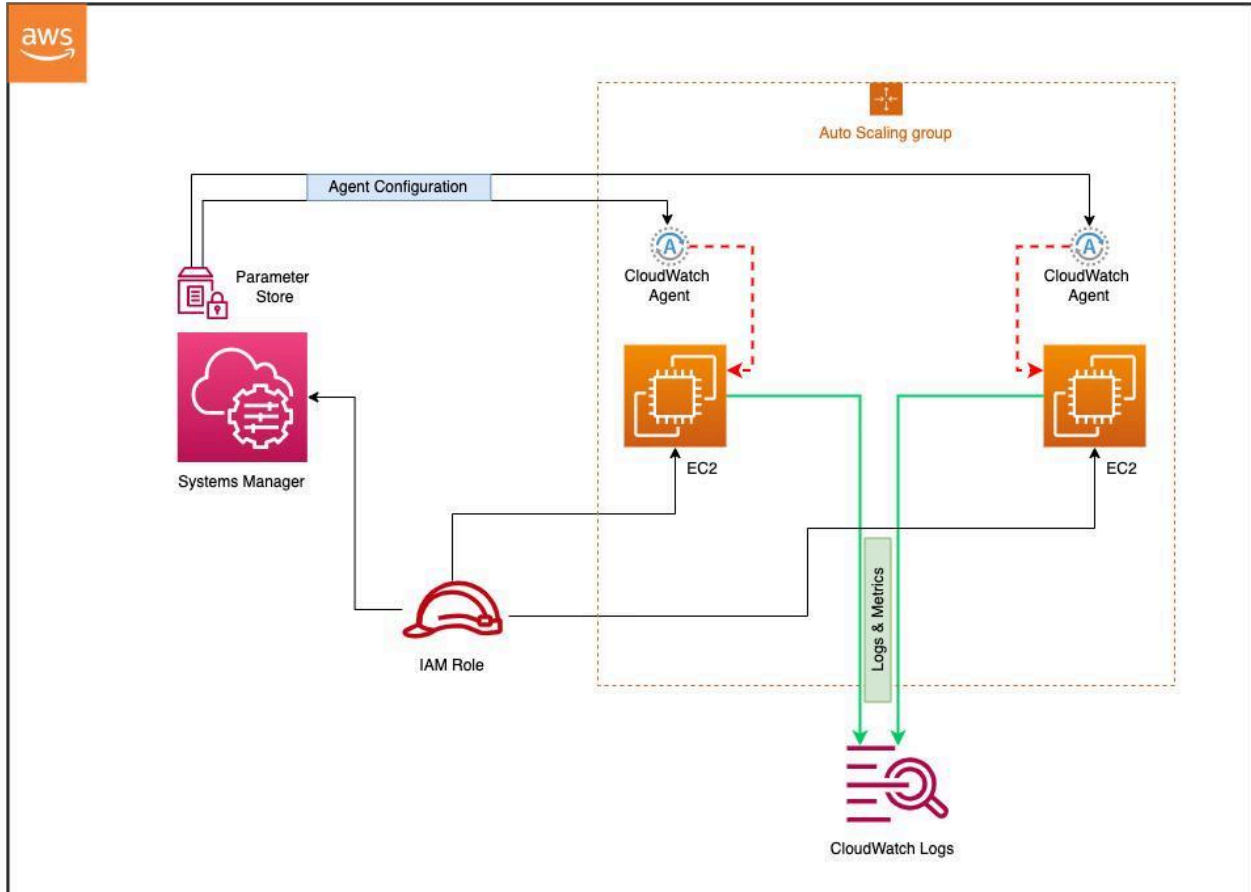
Core Idea

Observe → Detect → Alert → Act

3. CloudWatch Architecture

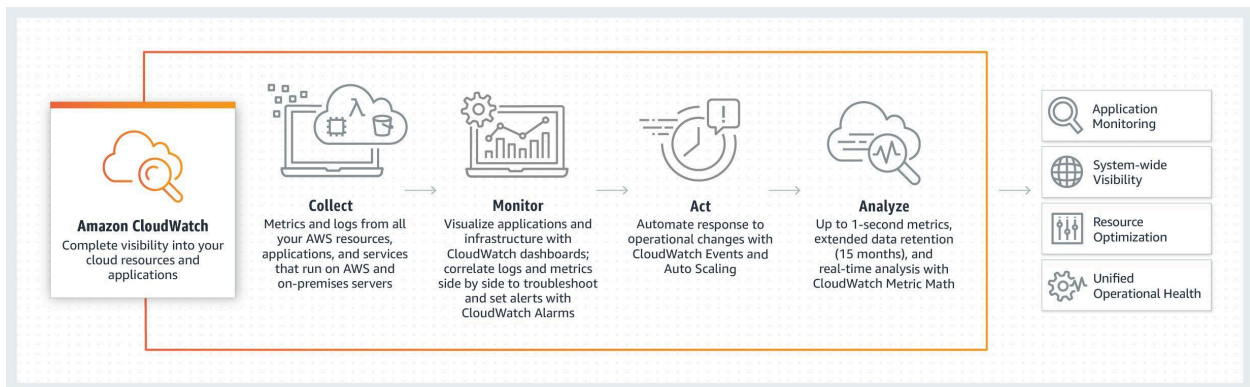
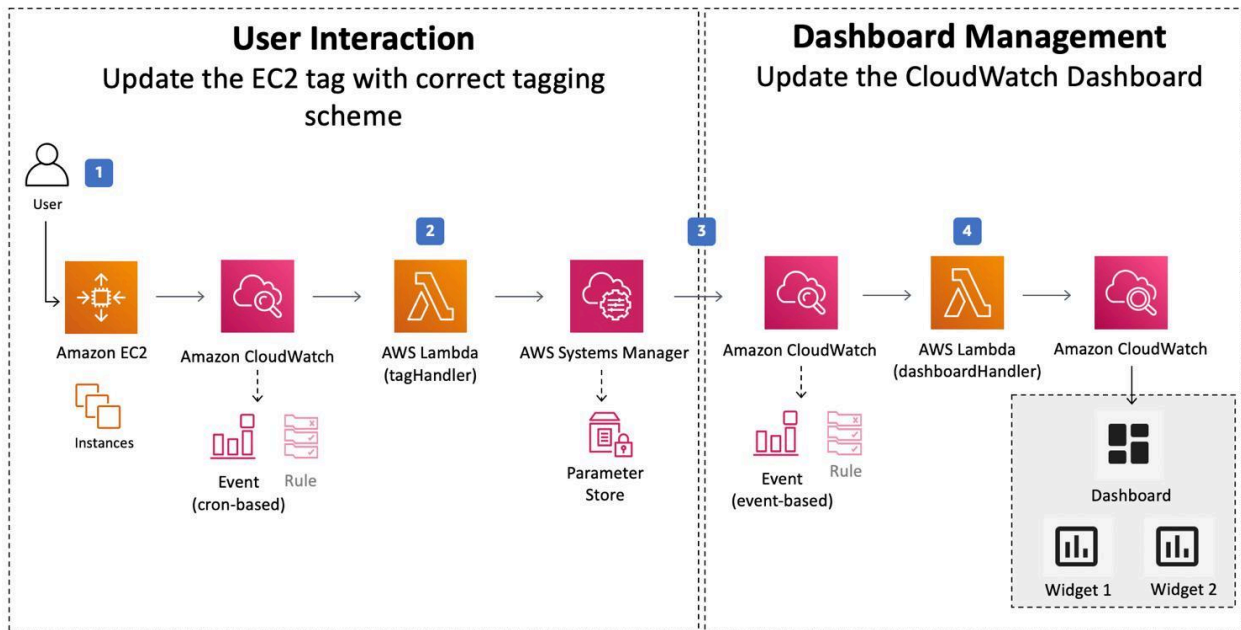






Key Differences Between Metrics, Logs, and Events

Feature	Purpose	Example
Metrics	Performance measurements of resources	CPU utilization, Disk I/O, Network In
Alarms	Notify or act based on metric thresholds	Alert if CPU > 80%
Logs	Detailed records of system/application events	Error logs, debug information
Events	Real-time triggers for resource changes	Notify on instance launch



4. Core Components (Must Know Deep)

◆ 1. Metrics

What:

Time-series numerical data

Examples:

Service	Metric
EC2	CPUUtilization
ALB	RequestCount
RDS	DBConnections

Key Concepts:

- Namespace → AWS/EC2
 - Dimensions → InstanceId
 - Period → 1 min / 5 min
 - Statistic → Average / Sum / Max
-
-

2. Logs

What:

Application & system logs stored centrally

Structure:

Log Group → Log Stream → Events

Real Example:

Log Group: /aws/ec2/app

Log Stream: i-12345

◆ 3. Alarms

What:

Trigger action based on metric threshold

Example:

CPU > 80% → Send alert

States:

- OK
- ALARM
- INSUFFICIENT_DATA

◆ 4. Dashboards

What:

Visual monitoring panel

Example:

- CPU graph
 - Request count
 - Error rate
-
-

◆ 5. Events (EventBridge Integration)

What:

Trigger actions based on events

Example:

EC2 stopped → trigger Lambda

5. Metrics Deep Dive (IMPORTANT)

Types:

◆ Default Metrics

Automatically provided by AWS

◆ Custom Metrics

Your application metrics

Example:

```
aws cloudwatch put-metric-data \  
--metric-name LoginCount \  
--namespace MyApp \  
--value 5
```

6. Logs Deep Dive

Use Cases:

- Application debugging
 - Security auditing
 - Centralized logging
-

EC2 Log Setup (Production)

Install agent:

```
sudo apt install amazon-cloudwatch-agent
```

Configure:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard
```

Start:

```
sudo systemctl start amazon-cloudwatch-agent
```

7. Alarms (Production Setup)

Example: CPU Alarm

- Metric: CPUUtilization
 - Threshold: > 70%
 - Duration: 5 minutes
-

Action:

- SNS notification
 - Auto Scaling
 - Lambda trigger
-
-

8. SNS Integration (Alerting)

Flow:

CloudWatch Alarm → SNS → Email / SMS

9. Real Production Architecture

EC2 → Metrics → CloudWatch → Alarm → SNS → Email

↓

Logs → CloudWatch Logs → Debugging

10. Real-World Scenarios

Scenario 1: High CPU

- Monitor CPU
 - Alarm triggers
 - Auto scale EC2
-
-

Scenario 2: Application Logs

- App crashes
- Logs pushed to CloudWatch

- Debug remotely
-
-

Scenario 3: Security Monitoring

- Unauthorized login attempts
 - Logged and alerted
-
-

11. Best Practices (Production)

Use Naming Standards

/dev/app/logs
/prod/api/logs

Set Retention Policies

Delete logs after 30/90 days

Use Structured Logs (JSON)

Enable Detailed Monitoring

Use Dashboards for visibility

12. Common Mistakes

- ✗ No log retention → high cost
 - ✗ No alarms → no alerting
 - ✗ Too many alarms → alert fatigue
 - ✗ Not monitoring logs
-
-

13. Interview Questions

Q1: Difference between logs and metrics?

Metrics = numbers

Logs = text/events

Q2: What triggers CloudWatch alarm?

Threshold breach on metric

Q3: How to monitor application logs?

Using CloudWatch Logs + agent

Q4: How to send alerts?

CloudWatch Alarm → SNS

CloudWatch Hands-On Lab

AWS CloudWatch Hands-On Lab (Beginner Friendly)

Objective

By the end of this lab, students will learn:

- How to monitor an EC2 server
 - How to get alerts when CPU is high
 - How to send alert emails using SNS
 - How to view logs from a server
 - How to debug issues using logs
-

Simple Understanding

Think like this:

EC2 (server) → CloudWatch (monitoring) → Alarm → SNS → Email
EC2 → CloudWatch Logs → Debugging

STEP 1: Launch EC2 Instance

Go to:

👉 Amazon EC2 → Launch Instance

Fill the details:

- Name: **cw-demo-ec2**
- AMI: Ubuntu / Amazon Linux
- Instance type: t2.micro

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags Info

Name: [Add additional tags](#)

Application and OS Images (Amazon Machine Image) Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Recents | My AMIs | **Quick Start**


Amazon Linux


macOS


Ubuntu


Windows


Red Hat


SUSE Linux


Debian

[Browse more AMIs](#)
Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.1 AMI
ami-0e12ff2d8d4655fe4 (64-bit (x86_64), uefi, preferred) / ami-0c54092ef0e77b87f (64-bit (x86_64), uefi)
Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Amazon Linux 2023 (kernel-6.1) is a modern, general purpose Linux-based OS that comes with 5 years of long term support. It is optimized for AWS and designed to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Amazon Linux 2023 AMI 2023.11.20260413.0 x86_64 HVM kernel-6.1

Architecture	Boot mode	AMI ID	Publish Date	Username
x86_64	uefi	ami-0e12ff2d8d4655fe4	2023-04-10	ec2-user

Summary

Number of instances Info

Software Image (AMI)

Amazon Linux 2023 AMI 2023.11...[read more](#)
ami-0e12ff2d8d4655fe4

Virtual server type (instance type)

t3.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

👉 Without this → logs will NOT work

STEP 2: Create SNS Topic (Email Alert System)

Go to:

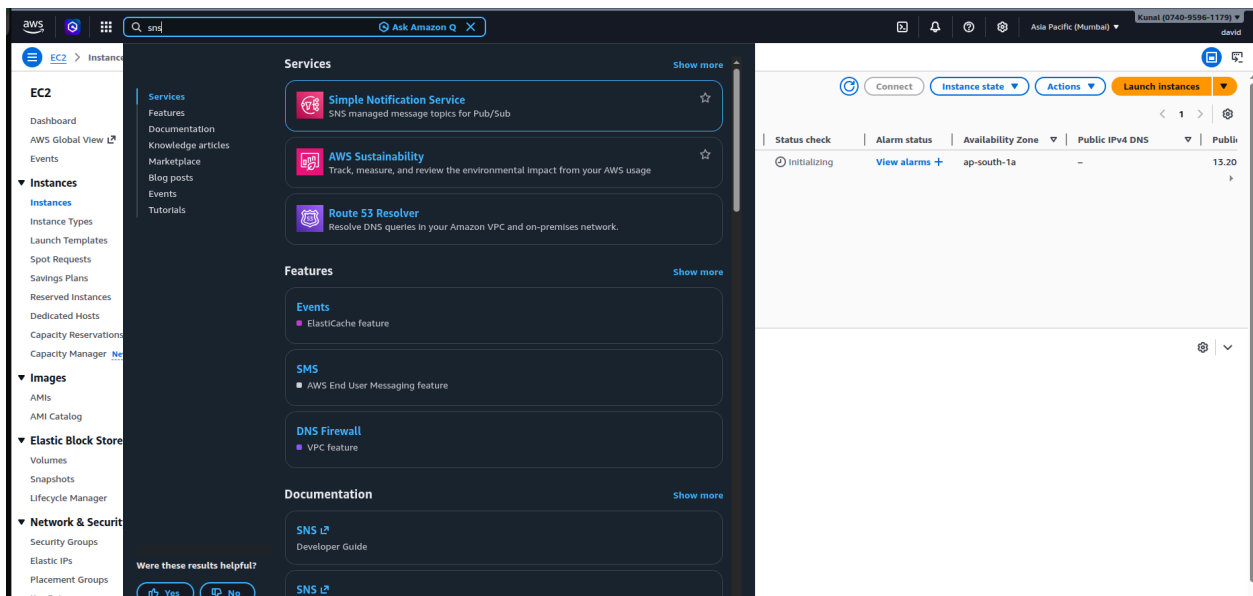
👉 Amazon SNS → Topics → Create

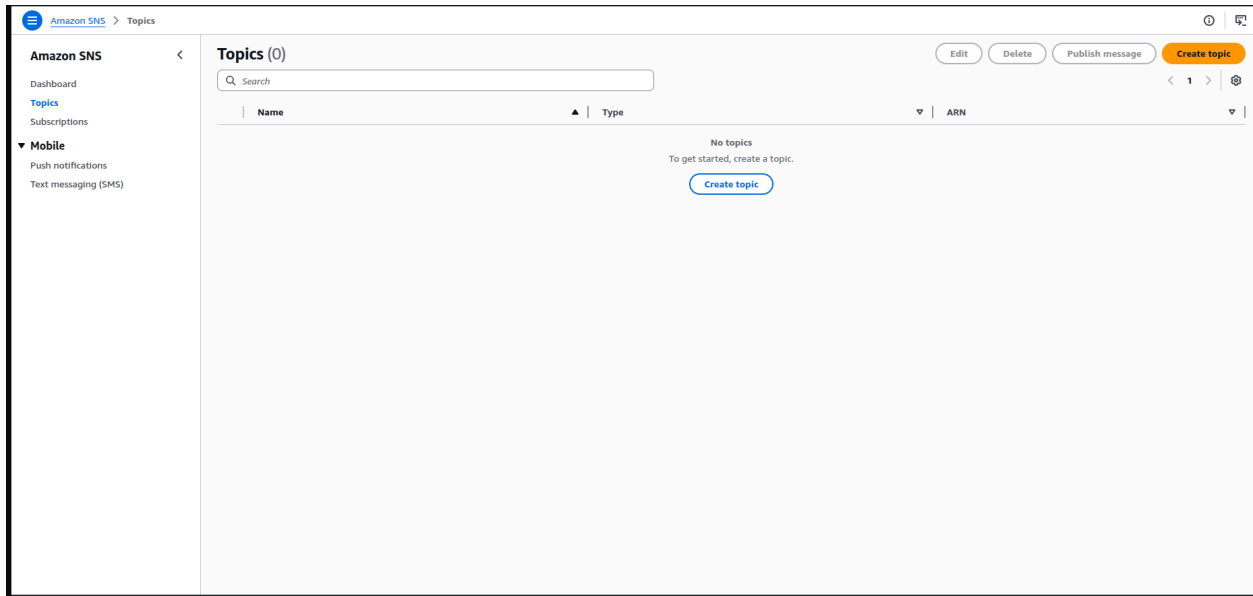
Select:

Type: Standard

Name:

cw-alert-topic





Amazon SNS > Topics > Create topic

Create topic

Details

Type [Info](#)
Topic type cannot be modified after topic is created

☐ **FIFO (first-in, first-out)**

- Strictly-preserved message ordering
- Exactly-once message delivery
- Subscription protocols: SQS

☒ **Standard**

- Best-effort message ordering
- At-least once message delivery
- Subscription protocols: SQS, Lambda, Data Firehose, HTTP, SMS, email, mobile application endpoints

Name

Maximum 256 characters. Can include alphanumeric characters, hyphens (-) and underscores (_).

Display name - optional [Info](#)
To use this topic with SMS subscriptions, enter a display name. Only the first 10 characters are displayed in an SMS message.

Maximum 100 characters.

► **Encryption - optional**

Amazon SNS provides in-transit encryption by default. Enabling server-side encryption adds at-rest encryption to your topic.

► **Access policy - optional** [Info](#)

This policy defines who can access your topic. By default, only the topic owner can publish or subscribe to the topic.

► **Data protection policy - optional** [Info](#)

This policy defines which sensitive data to monitor and to prevent from being exchanged via your topic.

Add Email Subscription

Click **Create Subscription**

Fill:

- Protocol: Email
- Endpoint: your email

Click **Create**

👉 Now check your email and **CONFIRM** subscription

The screenshot shows the 'Create subscription' page in the Amazon SNS console. The 'Details' section contains the following fields:

- Topic ARN:** A text input field containing 'arn:aws:sns:ap-south-1:074095961179:cw-alert-topic'.
- Protocol:** A dropdown menu with 'Email' selected.
- Endpoint:** A text input field containing 'gauravhalnawar1011@gmail.com'.

Below the 'Endpoint' field, there is a blue information box that reads: 'After your subscription is created, you must confirm it. info'.

There are two expandable sections below the details:

- Subscription filter policy - optional** (with an 'info' link): This policy filters the messages that a subscriber receives.
- Redrive policy (dead-letter queue) - optional** (with an 'info' link): Send undeliverable messages to a dead-letter queue.

At the bottom right, there are two buttons: 'Cancel' and 'Create subscription'.

The screenshot shows the 'cw-alert-topic' page in the Amazon SNS console. The 'Details' section displays the following information:

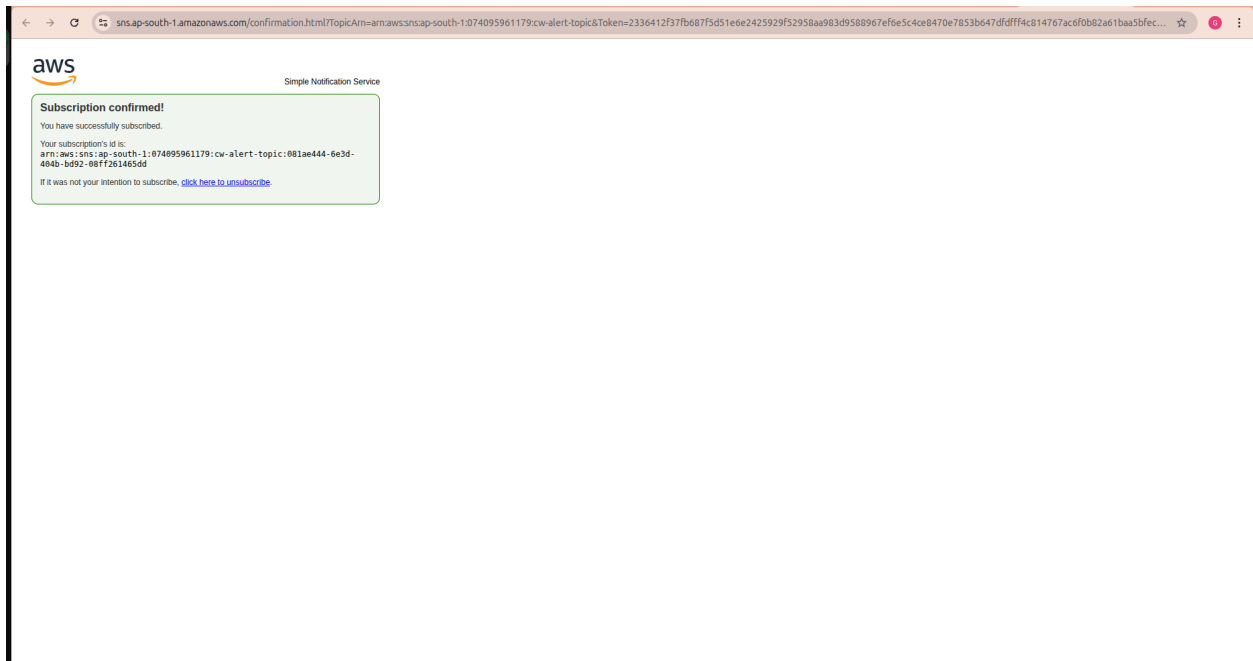
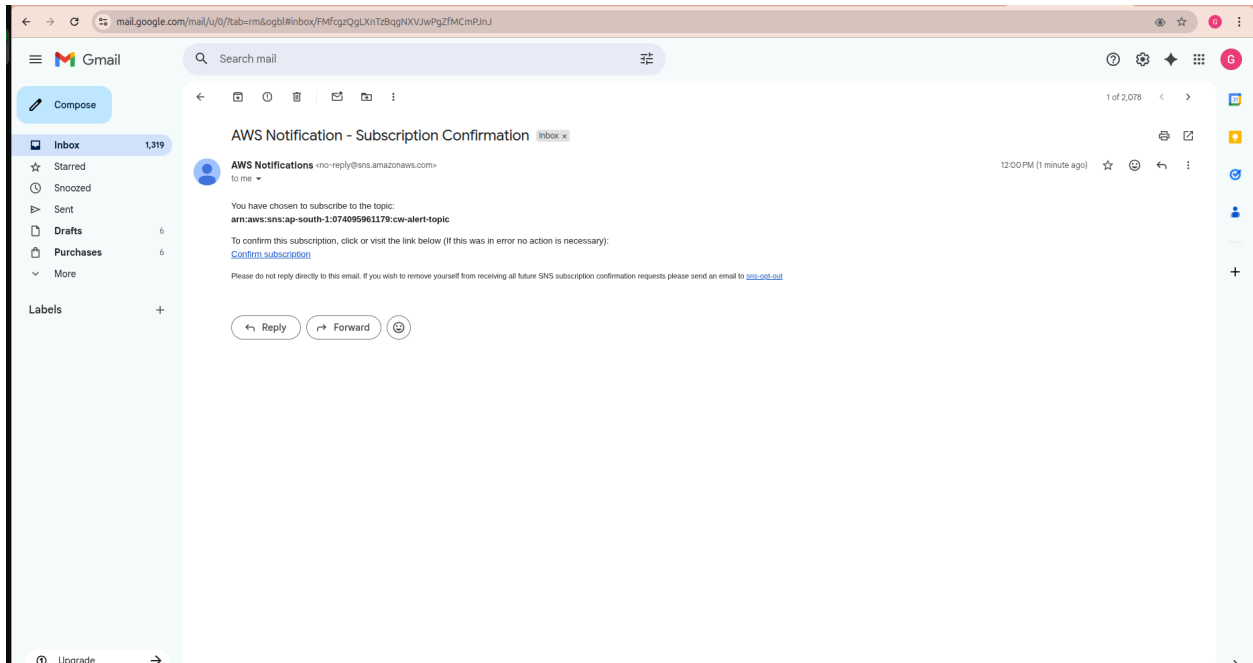
- Name:** cw-alert-topic
- ARN:** arn:aws:sns:ap-south-1:074095961179:cw-alert-topic
- Display name:** cw-alert-topic
- Type:** Standard
- Topic owner:** 074095961179

Below the details, there are tabs for 'Subscriptions', 'Access policy', 'Data protection policy', 'Delivery policy (HTTP/S)', 'Delivery status logging', 'Encryption', 'Tags', and 'Integrations'. The 'Subscriptions' tab is active.

The 'Subscriptions (1)' section shows a table with one subscription:

ID	Endpoint	Status	Protocol
Pending confirmation	gauravhalnawar1011@gmail.com	Pending confirmation	EMAIL

At the top right of the 'Subscriptions' section, there are buttons: 'Edit', 'Delete', 'Request confirmation', 'Confirm subscription', and 'Create subscription'.



The screenshot shows the Amazon SNS console interface. On the left is a navigation menu with 'Amazon SNS' at the top, followed by 'Dashboard', 'Topics', 'Subscriptions', and 'Mobile'. The main content area is titled 'cw-alert-topic' and includes buttons for 'Edit', 'Delete', and 'Publish message'. Below the title is a 'Details' section with fields for 'Name' (cw-alert-topic), 'ARN' (arn:aws:sns:ap-south-1:074095961179:cw-alert-topic), 'Display name' (cw-alert-topic), and 'Type' (Standard). The 'Topic owner' is listed as 074095961179. Below the details is a tabbed interface with 'Subscriptions' selected. The 'Subscriptions (1)' section includes a search bar and a table with one subscription. The table has columns for ID, Endpoint, Status, and Protocol. The subscription details are: ID: 081ae444-6e3d-404b-bd92-08ff261465dd, Endpoint: gauravhlnawar1011@gmail.com, Status: Confirmed, and Protocol: EMAIL. There are also buttons for 'Edit', 'Delete', 'Request confirmation', 'Confirm subscription', and 'Create subscription'.

STEP 3: Create CloudWatch Alarm

Go to:

👉 Amazon CloudWatch → Alarms → Create Alarm

Select Metric:

- EC2 → Per Instance Metrics
- Select your instance
- Choose:

CPUUtilization

CloudWatch

Alarms

CloudWatch

Alarms

Alarms (0)

Filter alarms

Quick filters

Clear selection

Create composite alarm

Actions

Create alarm

1

Name

State

Actions

Actions muted - new

Last state update (UTC)

Conditions

No alarms

No alarms to display

Read more about Alarms

Create alarm

CloudWatch

Alarms

Alarms (0)

Filter alarms

Quick filters

Clear selection

Create composite alarm

Actions

Create alarm

1

Name

State

Actions

Actions muted - new

Last state update (UTC)

Conditions

No alarms

No alarms to display

Read more about Alarms

Create alarm

Select metric

CPUUtilization

Percent

4.14%

4.07%

4%

03:30

03:40

03:50

04:00

04:10

04:20

04:30

04:40

04:50

05:00

05:10

05:20

05:30

05:40

05:50

06:00

06:10

06:20

06:30

CPUUtilization

Browse (18)

Multi source query

Graphed metrics (1)

Options

Source

=

Add math

Add query

☐

cw-demo-ec2

I-01F3888B44119...

EBSReadBytes

No alarms

☐

cw-demo-ec2

I-01F3888B44119...

EBSWriteOps

No alarms

☐

cw-demo-ec2

I-01F3888B44119...

EBSWriteBytes

No alarms

☐

cw-demo-ec2

I-01F3888B44119...

EBSIOBalance%

No alarms

☐

cw-demo-ec2

I-01F3888B44119...

EBSByteBalance%

No alarms

☐

cw-demo-ec2

I-01F3888B44119...

EBSReadOps

No alarms

☒

cw-demo-ec2

I-01F3888B44119...

CPUUtilization

No alarms

☐

cw-demo-ec2

I-01F3888B44119...

NetworkIn

No alarms

☐

cw-demo-ec2

I-01F3888B44119...

NetworkPacketsOut

No alarms

☐

cw-demo-ec2

I-01F3888B44119...

NetworkPacketsIn

No alarms

☐

cw-demo-ec2

I-01F3888B44119...

NetworkOut

No alarms

☐

cw-demo-ec2

I-01F3888B44119...

MetadataNoToken

No alarms

Cancel

Select metric

CloudWatch

Alarms

Create alarm

Step 1

Specify metric and conditions

Step 2

Configure actions

Step 3

Add alarm details

Step 4

Preview and create

Alarm recommendations

View details

Percent

70%

70

35.1%

0.142%

04:00

04:30

05:00

05:30

06:00

06:30

CPUUtilization

This alarm will trigger when the blue line goes above the red line for 1 datapoints within 1 minute.

Namespace

AWS/EC2

Metric name

CPUUtilization

Instanceid

i-01f3888b44119b1fa

Instance name

cw-demo-ec2

Statistic

Average

Period

1 minute

Threshold type

Static

Use a value as a threshold

Anomaly detection

Use a band as a threshold

Whenever CPUUtilization is...

Configure Alarm:

Threshold: Greater than 70%

Period: 1 minute (IMPORTANT for demo)

Datapoints: 1 out of 1

CloudWatch > Alarms > Create alarm

0.142%

04:0004:3005:0005:3006:0006:30

CPUUtilization

I-01f5888b44119b1fa

Instance name
cw-demo-ec2

Statistic
Average

Period
1 minute

Conditions

Threshold type

Static

Use a value as a threshold

Anomaly detection

Use a band as a threshold

Whenever CPUUtilization is...

Define the alarm condition.

Greater

> threshold

Greater/Equal

>= threshold

Lower/Equal

<= threshold

Lower

< threshold

than...

Define the threshold value.

70

Must be a number.

Additional configuration

Cancel

Next

CloudWatch > Alarms > Create alarm

Step 1

Specify metric and conditions

Step 2

Configure actions

Step 3

Add alarm details

Step 4

Preview and create

Configure actions

Notification

Alarm state trigger

Define the alarm state that will trigger this action.

In alarm

The metric or expression is outside of the defined threshold.

OK

The metric or expression is within the defined threshold.

Insufficient data

The alarm has just started or not enough data is available.

Send a notification to the following SNS topic

Define the SNS (Simple Notification Service) topic that will receive the notification.

Select an existing SNS topic

Create new topic

Use topic ARN to notify other accounts

Send a notification to...

cw-alert-topic

Only topics belonging to this account are listed here. All persons and applications subscribed to the selected topic will receive notifications.

Email (endpoints)

gauravhainawar1011@gmail.com - View in SNS Console

Add notification

Lambda action

Add Lambda action

Auto Scaling action

Add Auto Scaling action

High CPU Alert – EC2 Instance

Metric: CPUUtilization > 70%

Instance: cw-demo-ec2

What this means

CPU usage on the EC2 instance is above normal threshold. This may indicate high load, inefficient processes, or potential application issues.

What to check

- Check running processes ([top](#), [htop](#))
- Identify high CPU-consuming processes
- Verify application health
- Check recent deployments or traffic spikes

Immediate action

- Restart problematic service if required
- Scale up instance or enable Auto Scaling
- Kill unwanted processes if safe

Impact

- Application slowdown
- Increased response time
- Possible downtime if not handled

Owner

DevOps Team

Severity

High

STEP 4: Trigger Alarm (Live Demo)

Connect to EC2 using:

- SSH OR
 - SSM Session Manager
-

Run this command:

yes > /dev/null &

STEP 5: Install CloudWatch Agent (Logs)

Inside EC2:

Install agent:

```
sudo apt update  
sudo apt install amazon-cloudwatch-agent -y
```

Run setup:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard
```

Select:

- Collect logs → YES
- Log file:

```
/var/log/syslog
```

Start agent:

```
sudo systemctl start amazon-cloudwatch-agent
```

 **Screenshot 10: Agent Installation**

STEP 6: View Logs in CloudWatch

Go to:

👉 CloudWatch → Logs → Log Groups

Open:

/var/log/syslog

 **Screenshot 11: Log Group View**

STEP 7: Generate Logs

Run inside EC2:

logger "CloudWatch test log entry"

👉 Refresh CloudWatch logs

👉 You will see log entry

 **Screenshot 12: Log Entry Visible**

STEP 8: Create Dashboard

Go to:

 CloudWatch → Dashboards → Create

Add:

- CPU graph
 - Logs
-

 Screenshot 13: Dashboard

WHAT STUDENTS LEARN

Feature	What it means
Metrics	Monitor system (CPU, memory)
Alarm	Get alert when something is wrong
SNS	Send notification (email)
Logs	See what happened inside system
Agent	Sends logs to CloudWatch

COMMON MISTAKES

✗ Forgot IAM role
👉 Logs will NOT work

✗ Used SNS FIFO topic
👉 Email will NOT work

✗ Period set to 5 min
👉 Alarm delayed

✗ CPU not high
👉 Alarm won't trigger



REAL-WORLD USE CASE

Example:

At night, your server crashes:

CPU high → Alarm → Email → Check logs → Fix issue

👉 This is how companies monitor systems



INTERVIEW QUESTIONS

Q1: How do you monitor EC2?

Using CloudWatch metrics

Q2: How do you get alerts?

CloudWatch Alarm + SNS

Q3: How to debug server issues?

Using CloudWatch Logs

Q4: Why CloudWatch Agent?

To send logs from EC2 to CloudWatch



Final Note

This lab is not about clicking buttons.

It's about understanding:

Monitor → Alert → Investigate → Fix

If you want next:

- 🔥 Same lab in CLI version
- 🔥 Add Auto Scaling based on alarm
- 🔥 Real production monitoring setup

Just tell me.

SNS

```
aws sns create-topic --name cw-final-safe --region ap-south-1
```

```
aws sns subscribe \  
  --topic-arn <arn> --protocol email \  
  --notification-endpoint gauravhalnawar1011@gmail.com
```

```
aws sns list-subscriptions-by-topic --topic-arn  
"arn:aws:sns:ap-south-1:074095961179:cw-final-safe"
```

```
aws sns publish \  
  --topic-arn "arn:aws:sns:ap-south-1:074095961179:cw-trace-now" \  
  --subject "Test Alert" \  
  --message "SNS is working successfully"
```

Tab 21

Tab 22


```
docker login -u gauravhalnawar123
```

```
dckr_pat_J-7ref8JiYJwWHE9lwji2s9KweI
```